scientific reports

Check for updates

OPEN

Fast highway abandoned object detection via block-based multi-group foreground extraction

Dian Liu[1], Haoxiang Wang[1], Xiujie Zhang[2], Xingming Zhang[1] & Yang Liu[3]✉

Abandoned objects on highways pose a significant risk of causing severe traffic accidents. Current abandoned object detection technologies are limited by hardware constraints and real-time processing requirements in complex highway environments. To address these challenges, we propose a Universal foreground extraction framework consisting of Block-based frame selection and Multi-Group foreground Detection, called UBMG, which effectively alleviates false alarms caused by either illumination changes on the highway, traffic targets, or road markings. Specifically, two modules of block preprocessing and adaptive size filtering are first designed, followed by a static target matching module, to extract candidate targets. To accurately distinguish abandoned objects from other entities, a candidate verification strategy is proposed involving traffic target elimination, road noise elimination, and trajectory discrimination. This framework can seamlessly integrate with existing pixel-wise foreground detection algorithms and demonstrate high efficiency in practical applications. In addition, we have established a comprehensive video dataset of highway abandoned objects, named HAO, under various conditions (e.g.,lighting conditions, camera movements, and object occlusions) for thorough evaluation. Extensive experiments on HAO and public ABODA datasets demonstrate that the UBMG framework can perform robust and real-time detection of abandoned objects, especially outperforming the state-of-the-art methods in the HAO dataset and showing good performance on ABODA.

Keywords Abandoned object detection, Highway, Foreground extraction, Trajectories tracking, Real-time processing

Discarded objects or debris from passing vehicles on highways pose severe hazards to traffic safety. An intelligent obstacle avoidance and alert system on highways can significantly decrease the incidence of traffic mishaps and enhance the operational efficiency of highways. However, existing abandoned object detection research within video surveillance systems focuses on scenes where pedestrians frequently appear, especially in public and crowded places such as airports, buildings, or railway stations[1], with scant attention given to highways involving vehicle traffic. In addition, publicly available datasets of abandoned objects usually contain objects like trash, luggage, and other discarded items in indoor/outdoor areas[1], while neglecting specific objects in highway scenarios. Therefore, effective solutions and comprehensive benchmarks of highway abandoned object detection are crucial for ensuring the safety of passengers' lives and property, especially in the context of the rapid development of autonomous driving.

Current abandoned object detection methods comprise two principal components: foreground detection and object validation. The extraction of foreground objects is primarily achieved through background subtraction, utilizing background modeling algorithms. Predominant techniques include single-background models[2] and dual-background models[3], which exhibit sensitivity to dynamic background changes, camera jitter, and the appearance of ghost images[4]. designed a shadow detection framework through foreground detection and cast shadow detection to reduce the negative effects caused by the cast shadows of vehicles in traffic surveillance video analysis. The object validation phase principally considers pedestrian-related factors to identify objects such as garbage, luggage, and pedestrian-abandoned items. A variety of techniques are employed to minimize false detections, focusing on large objects with regular outlines, such as edge clustering[5], pedestrian detection[6], backtracking[7], similarity comparison[8–10], and deep neural networks[11]. Recently, some deep learning research has been conducted on improving the detection performance against challenging outdoor conditions on highways,

[1]Computer Science and Engineering, South China University of Technology, Guangzhou 510006, China. [2]Guangzhou Run One Traffic Information Co., Ltd, Guangzhou 510006, China. [3]Center for Machine Vision and Signal Analysis, University of Oulu, Oulu 90570, Finland. ✉email: yang.liu@oulu.fi

but they highly rely on GPU acceleration or carefully optimized algorithms[12]. For example[13], utilized YOLO to detect individuals with luggage, while[14] proposed a deep learning framework tailored for detecting outdoor hand luggage as a proxy for abandoned objects[15]. proposed an efficient object localization and classification method for edge devices through seamlessly blending the strengths of semantic segmentation and object detection. However, these deep models usually require extensive training datasets, which are unsuitable for highway contexts due to unpredictable object categories. Additionally, they do not have universality and cannot be seamlessly integrated into existing systems and devices, increasing implementation costs.

In general, there are several issues when deploying abandoned object detection systems on highways. Firstly, methods need to be highly efficient, enabling them to detect multiple video streams simultaneously with limited hardware resources[16]. Common hardware components on highways are roadside units with elementary computing power, either based on CPUs or GPUs. Meanwhile, since there are thousands of surveillance cameras along the highway, a universal detection framework is required that can flexibly integrate with various detectors and devices. Secondly, highway scenes are susceptible to complicated illumination variations, thereby degrading detection performance[4]. For example, sudden lighting changes in daytime environments can easily cause background modeling failures, while lighting changes in nighttime or tunnel scenes pose a significant challenge to both object detection models and background models. Finally, small traffic targets or lane markings at the far end of highways are prone to triggering false alarms of littering due to the wide range of monitoring areas[17].

In this study, we propose a universal framework for highway abandoned object detection by performing block-based multi-group foreground extraction, named UBMG, which significantly reduces computational burden while preserving the detection accuracy. Following the framework, we revolutionize the detection procedure in terms of block-based frame selection, multi-group foreground detection, static target matching, and candidate verification, enabling robust and fast abandoned object detection in complex highway scenarios. This framework can be used for the foreground detection process in general scenarios. Besides, a comprehensive video dataset of highway abandoned objects (HAO) is established to provide sufficient training data and extensive experimental validation.

The main contributions of this paper are outlined as follows:

- The proposed block-based frame selection approach and multi-group foreground detection method can significantly reduce the impact of illumination variations in different highway scenarios, including day vs. night and indoor vs. outdoor, and filter suspicious foreground target sizes without relying on fixed thresholds, respectively.
- We design strategies of static target matching and candidate verification that can effectively reduce false positives caused by pavement disturbances and stationary traffic objects, thereby improving the overall detection performance.
- The constructed highway abandoned object video dataset, involving 103 fully annotated sequences across three distinct scenarios, provides valuable resources for training and evaluating models of abandoned object detection. *For access requests, please contact the first author, Mr. Dian Liu (csdianl@mail.scut.edu.cn).*
- Extensive experiments on the collected highway dataset and the public general abandoned object dataset demonstrate that our proposed UBMG framework strikes a trade-off between detection performance and computational costs, achieving precise and fast detection and outperforming the state-of-the-art methods.

The following sections are arranged as follows: Section 2 introduces the relevant work; Section 3 introduces the universal UBMG foreground extraction framework for highway abandoned object detection; Section 4 introduces the collected HAO video dataset and presents the experimental evaluation of UBMG; Section 5 presents the conclusion of this paper.

Related work

Anomalous object detection

The detection of abandoned objects on highways can be considered a subset of anomalous object detection, particularly crucial within the context of autonomous driving. In recent years, remarkable strides have been achieved in the realm of object detection methodologies, with an array of studies earmarking their application towards the detection of anomalous objects. Leading-edge techniques, including YOLO[18], Faster R-CNN[19], and SSD[20], have demonstrated adeptness in delineating object boundaries via bounding boxes for pre-defined categories. Kraft et al.[21] introduced a cost-effective approach leveraging drones to amass a dataset of refuse, thereby crafting a platform for garbage detection. This system, underpinned by a deep neural network, is adept at identifying discarded waste and autonomously formulating collection trajectories. Cortesi et al.[22] unveiled an enhancement to object detection paradigms based on the Faster R-CNN framework, integrating a feature pyramid network. This refinement was achieved by amalgamating disparate layers through multiplication, harnessing focal loss to rectify anchor imbalances, and introducing an attention module to augment the entire network's performance, culminating in a litter detection accuracy surpassing 98%. In a parallel vein, Yang et al.[23] put forward a pioneering technique for the automated discernment and extraction of litter from Terrestrial Laser Scanning (TLS) 3D point clouds, highlighting commendable resilience across a spectrum of natural environments, litter types, vegetation densities, and environmental perturbations. Pfeiffer et al.[24] embarked on utilizing both YOLOv5 and Faster R-CNN algorithms to pinpoint objects along Malta's coastline through drone-captured footage. Zhou et al.[25] innovated within the tracking domain by integrating a heat map into the CenterTrack model, which processes the thermal signatures of detected objects across sequential frames to ascertain the directional shifts of targets. Peicheng Shi et al.[26] proposed a Transformer Feature Fusion Encoder (TFFEn) for joint depth–image modeling, a deformable convolution-based enhancement (FEDC) guided by depth confidence to better handle truncated objects, and a Depth-perception NMS (Dp-NMS) that fuses classification and depth

confidences to improve proposal ranking. Xinlong Dong et al.[27] introduced a CNN-transformer adaptive feature fusion network that uses a convolutional transformation adaptive fusion kernel to adaptively fuse local CNN and global Transformer features and an encoder-decoder to propagate this fused information across stages. At the same time, research on high performance algorithms for small object detection has made progress[12]. Amudhan et al.[16] proposed a novel hypermetropic Convolutional Neural Network that balances performance and cost to improve small object detection; Haotian Liu et al.[28] proposed a knowledge distillation framework with masked scale-aware feature distillation and local attention distillation to enhance small-model performance, particularly for improving the detection of tiny objects with high accuracy and efficiency. However, a critical gap remains in these methodologies' applicability to highway scenarios, particularly due to the ambiguous nature of abandoned object categories. This limitation underscores the necessity for specialized approaches tailored to highway abandoned object detection, where conventional methods may falter in efficacy.

Detecting objects that are not present in the training dataset, or those belonging to unknown categories, is a significant challenge in the domain of highway abandoned object detection. Lis et al.[29] introduced a method utilizing a disparity network to highlight differences between original input frames and re-synthesized images from semantic maps. While disparity-based approaches offer more reliability compared to uncertainty estimation methods, they are also prone to a higher rate of false positives. Building on this, Giancarlo et al.[30] proposed a novel anomaly segmentation framework that leverages both the uncertainty inherent in semantic maps and the dissimilarity between original and re-synthesized images. This method achieves notable enhancements in the detection of anomalous objects within autonomous driving scenarios, albeit at the expense of increased computational demand due to the necessity for pixel-wise analysis. Nayak et al.[31] contributed a structured and exhaustive review of the most recent advancements in video anomaly detection, with a particular focus on applications of deep learning. They introduced a graphical classification technique tailored for video anomaly detection that utilizes deep learning, marking a significant step forward in the field. Similarly, Vojir et al.[32] presented an innovative approach tailored for autonomous driving contexts, which calculates anomaly scores for individual pixels based on discrepancies in road structure similarity, leveraging a combination of a reconstruction module and a semantic segmentation network to identify unknown objects. Alternatively, Bogdoll et al.[33] explored the integration of various sophisticated detection models to analyze data from both LiDAR and cameras in anomalous object detection. Nevertheless, such methods are less effective in the specific case of detecting abandoned objects on highways. This reduced efficacy can be attributed to limitations in image quality and the distance from which monitoring devices can operate on highways.

Abandoned object detection

As mentioned above, the detection of abandoned objects in surveillance video streams is a critical task that encompasses two primary phases. In the initial phase, researchers have explored various approaches to segregate foreground entities from the background. St-Charles et al.[34] adopted the classical method of single-background subtraction to delineate foreground objects. Lin et al.[35] enhanced this approach by incorporating a finite state machine that captures temporal transitions of static foreground pixels. Despite these advancements, methods based on single or dual-background subtraction are prone to errors caused by ghost effects, changes in lighting, camera jitter, and other disturbances. To mitigate these limitations, some studies have shifted towards more sophisticated background modeling techniques. Ilias et al.[5] introduced a method that relies on edge background subtraction and edge clustering to extract stable edge masks, which is demonstrated to be resilient to occlusion and lighting variations. Concurrently, the advent of deep learning has spurred innovations in this domain, with Kim et al.[14] proposing the CNN-HLDNet, a deep learning framework tailored for the detection of outdoor hand luggage as a proxy for abandoned objects. These methods primarily focus on addressing the challenges posed by illumination variations and occlusions in indoor settings but often fall short in outdoor environments.

The second phase involves the verification of whether an extracted object has indeed been abandoned. Yun et al.[36] proposed a new framework to detect dumping action from cameras, in which background subtraction and human joint estimation are utilized to model the distance relationship between pedestrians and objects to determine dumping action. Sarker et al.[37] introduced a real-time monitoring system that employs shadow removal and background subtraction in human and object detection, integrating Histogram of Oriented Gradients (HOG) and Support Vector Machine (SVM) to recognize dumping actions. Santad et al.[13] utilized YOLO to detect individuals with luggage and track their movements to identify suspicious activities indicative of abandonment, such as placing luggage down and walking away. Shyam[38] trained a detection network with backpacks and handbags as a training dataset to detect abandoned objects. Mohiu et al.[39] applied a frame-differential technique to ascertain the historical location of luggage, enabling tracking of moving objects. However, the unique challenges posed by highway environments, such as the extended distances of monitoring scenes (often exceeding one hundred meters), dynamic backgrounds, and frequent camera jitter due to passing vehicles or strong winds, significantly complicate the application of existing methods. These conditions lead to a high rate of false positives, complicating subsequent analysis. Additionally, the diverse range and size of objects typically abandoned on highways, ranging from small bottles and paper pieces to tires and vehicle components, necessitate real-time detection solutions tailored to these specific contexts. The existing methodologies, primarily developed for urban or indoor scenarios, are ill-equipped to address the nuances of highway environments, highlighting the need for innovative approaches in this area.

Fast highway abandoned object detection via UBMG framework

Overview

In this section, we propose a universal foreground extraction framework, involving block-based frame selection and multi-group foreground detection, and specifically refine it for efficient detection of abandoned objects on highways. The complete method includes four integral components, as shown in Fig. 1. Specifically, the block-

based frame selection module contains road masking, block-based extraction, and block preprocessing to reduce computational load. Subsequently, blocks among different frames are grouped and are subjected to foreground detection, respectively, in terms of background modelling, foreground mask, morphological operation, and adaptive size filtering for multi-group, to identify suspicious targets. Then, a static target matching strategy is applied to obtain target candidates by accommodating the possibility of multiple matches per target, enhancing the precision of object identification. Finally, non-abandoned objects within target candidates are discarded based on traffic object classification and binary road noise removal, while trajectory discrimination facilitates the distinction of real targets by predicting transition states of objects, allowing for the meticulous detection and analysis of abandoned objects in highway scenarios.

Block-based frame selection

Foreground detection is a key component to model every pixel of the image and update the state of all pixels iteratively to determine whether they are foreground or background in traditional abandoned object detection algorithms[40]. According to our preliminary experiments, focusing only on updating pixels with detected changes and disregarding stable pixels significantly reduces computational load while preserving detection accuracy, because comparing pixel changes demands significantly less computational power than modeling background variations. Following this observation, we propose a block-based frame selection module to ensure low computational costs and high efficiency for highway scenarios. The module includes three parts: road masking, block-based extraction, and block preprocessing.

Road masking aims to extract images of road areas, further reducing computational requirements. Considering that highway monitoring equipment is typically installed in fixed locations, the monitored areas remain constant. Thus, we adopt Deeplabv3+[41] for extracting road regions from video frames, which processes input frames to yield a binary image delineating the road area, as shown in Fig. 2. This approach significantly decreases the computational load and minimizes the likelihood of false alarms.

Block-based extraction divides each frame into blocks and applies a block update strategy to decide their status. Figure 3 presents an example of the workflow. The input frame I'_{N+1} is divided into block-based frames B_{N+1}, where blocks that remain unchanged or contain no foreground in the last foreground frame are masked out in subsequent foreground detection steps.

To achieve this, an extension block is designed to enhance the block-based frame extraction algorithm, specifically targeting the detection of small, abandoned objects spanning multiple blocks, which existing methods often overlook due to minimal variation. Formally, a block region is defined as (x, y, w, h) with a minimum object detection size d required by the system. The corresponding extension-block is defined as $(x, y, w+2d, h+2d)$, confined within the boundaries of the road region. A block within the current frame is marked as changed if the pixel variation within its corresponding extension-block or the number of foreground pixels in the last frame extension-block exceeds the threshold described in Formula (1). Continuous detection is required for blocks with foreground until they blend into the background. The extension block is as illustrated in Fig. 4.

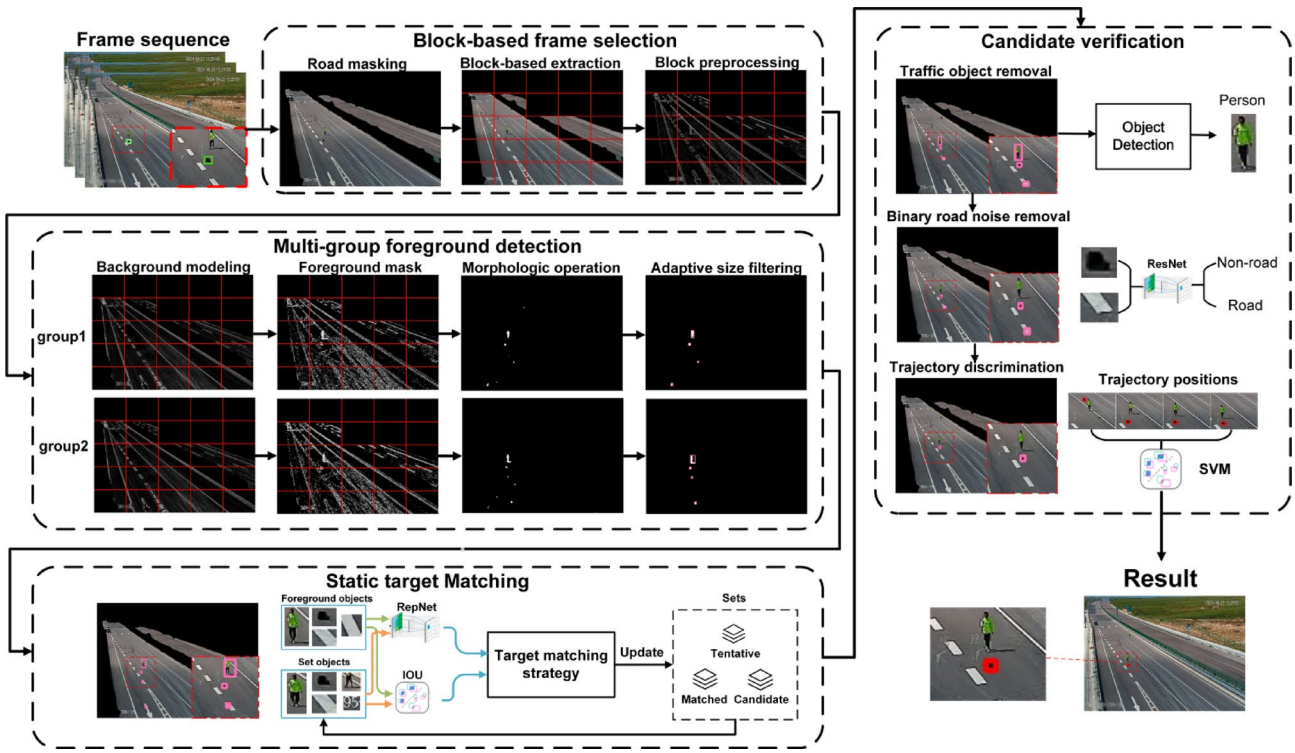


Fig. 1. The proposed UBMG framework for highway abandoned object detection.

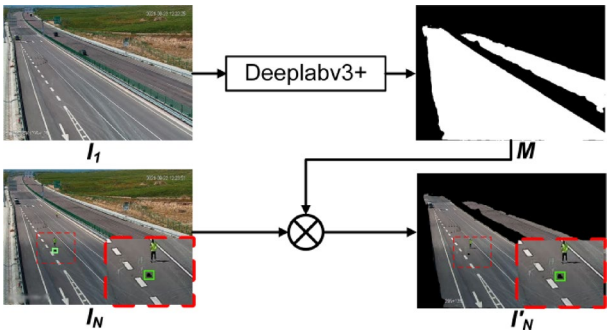


Fig. 2. The pipeline of road masking. The first frame I_1 is used to extract road region mask M according to Deeplabv3 + algorithm, then all the frames I_N are masked with M to extract road images I'_N.

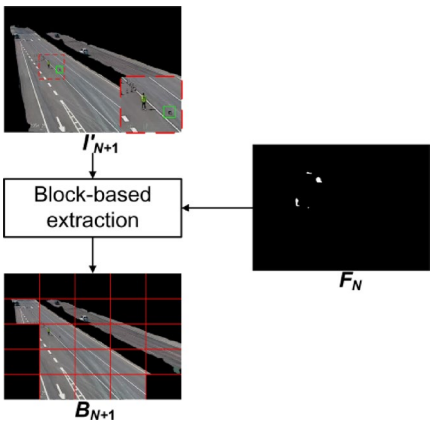


Fig. 3. The workflow of block-based extraction. It is a daytime scene on the highway, where a box is discarded on the road. I'_{N+1} is the image of road masking. F_N is the foreground mask of the last frame I'_N. The image I'_{N+1} is divided into blocks. And for each block, the number of pixels that have changed and the number of foreground pixels in last frame block are counted. If both numbers are smaller than the thresholds which are determined by the minimum detection target size required by the system, the block is marked as unchanged in black. Multiple black blocks indicate that there are many pixels in the highway scene that do not need to be updated.

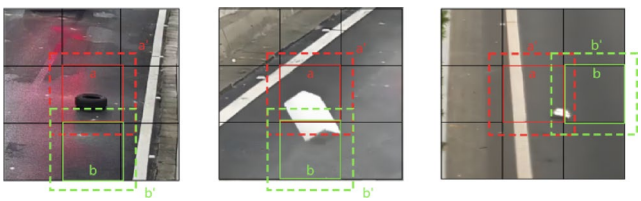


Fig. 4. Different locations of abandoned objects in blocks of the image. Extension-blocks (**a`**) and (**b`**) correspond to Block (**a**) and (**b**) separately drawn with red and green. The first image shows a tire located only in block (**a**). The second and third images show the abandoned objects situated amidst multiple blocks which make blocks (**a**) and (**b**) marked as changed.

The block-based extraction algorithm, detailed as Algorithm 1, utilizes a structure called 'BlockStruct' to track the state of each block for pixel comparison. The input image is divided into *nRow*nCol* blocks. These blocks undergo *nExtracted* consecutive foreground detections without the application of the *BlockChange* function if they are judged as changed, defined in Formula (1):

$$BlockChange(R, I_i) = \begin{cases} True, \sum_{x,y \in R} II(\Delta I_i^Y(x,y) > \theta) > d * dor \sum_{x,y \in R} II(F_i(x,y) > 0) > \alpha * d * d \\ False \cdot otherwise \end{cases} \quad (1)$$

Input: frame sequence$\{I_1, I_2, I_3..., I_n\}$,$nRow$,$nCol$,$nExtracted$
Output: block frame sequence

```
Struct BlockStruct {
curFrame ;        // the frame corresponding to current block
eNum ;             // the number of times extracting required
bRegion;                          // block region as [x,y,w,h]
extRegion;}     // extend-block region as [x,y,w+2s,h+2s]
bs=init(nRow,nCol);              // init as BlockStruct[nRow][nCol]
for I_i to {I_1, I_2, ..., I_n} do
    for i ← 0 to nRow do
        for j ← 0 to nCol do
            if bs[i][j].eNum > 0 then
                bs[i][j].eNum-=1;
            else if BlockChange(bs[i][j],I_i) is True then
                bs[i][j].eNum=nExtracted;
            else continue;;
            bs[i][j].curFrame=I_i;
    get block-based frame with eNum > 0 to extract foreground
```

Algorithm 1. Pseudo-code of block-based extracting.

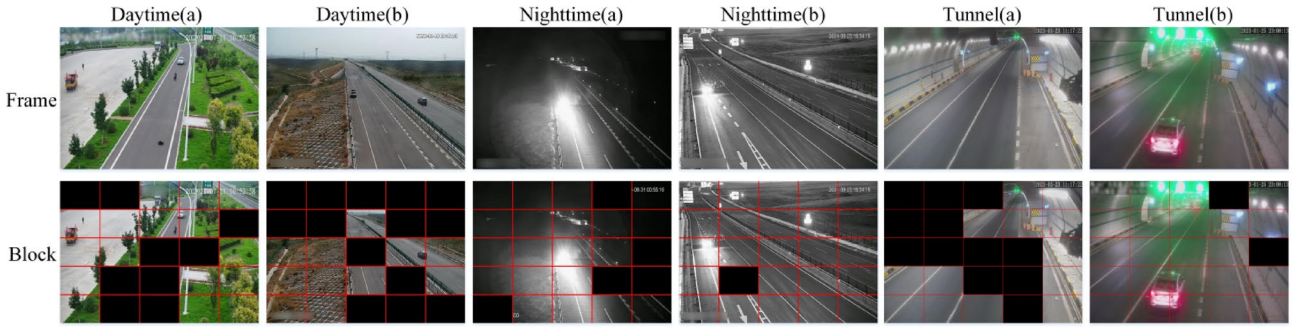


Fig. 5. comparison of block-based frame and original frame in different scenes. Black blocks mean these blocks do not need to update status. In daytime (**a**) a tire was discarded by a vehicle that had already stopped at the top of the screen and merged into the background. In daytime (**b**) the vehicle was driving on the road while another vehicle was parked on the emergency lane. In Nighttime (**a**) the vehicle with its lights war is parked at night. In Nighttime (**b**) vehicles were traveling in the distance. In Tunnel (a) there existed no vehicle in tunnel. In Tunnel(b) a vehicle was traveling in the scene. Daytime (**b**), Nighttime (**b**), Tunnel (**b**) show that there are still unchanged blocks when vehicles pass.

where *d* is the minimum detection size required by the system. Notably, brightness change ΔI_i^Y is used as one criterion for distinguishing content changes. The parameter θ is used to control the sensitivity of detecting lighting. If the number of pixels with a brightness change exceeds *d*d*, the block is considered to contain content changes. The second criterion involves the count of foreground pixels within the block, with an indicator function $\mathbb{I}$returning 1 if the condition is met. α is a parameter equal to the square of the mean of the normal distribution of size ratios described in Sect. 4.3. Given the considerable spatio-temporal variability of highway traffic, block-based frame extraction can efficiently identify static blocks that do not require updates, thus enhancing foreground detection efficiency. Figure 5 illustrates experimental results during daytime, nighttime, and within a tunnel.

Block preprocessing is additionally introduced to alleviate the impact of sudden changes in lighting on highways, as presented in Fig. 6. Contrast Limited Adaptive Histogram Equalization (CLAHE)[42] is applied to perform histogram equalization on each block, enhancing image contrast while suppressing noise, thereby ensuring that tiny, abandoned objects are not excessively diminished. To address illumination invariance, the brightness of individual blocks is processed by converting color scheme to *YCrCb*, where histogram equalization is applied to the Y channel. Then, the channels are converted back to RGB for the next processing steps as $b_{ir}^*, b_{ig}^*, b_{ib}^*$. For each channel, morphological operations are employed to remove noises as $b_{ibm}^*, b_{igm}^*, b_{irm}^*$ then *Absdiff* operation calculates the absolute value of $b_{ibm}^*, b_{igm}^*, b_{irm}^*$ and the corresponding channel $b_{ir}^*, b_{ig}^*, b_{ib}^*$. After the absolute values are normalized to [0,255], they are merged to obtain the final block image b_i' .

The morphological operation is described in Formula (2):

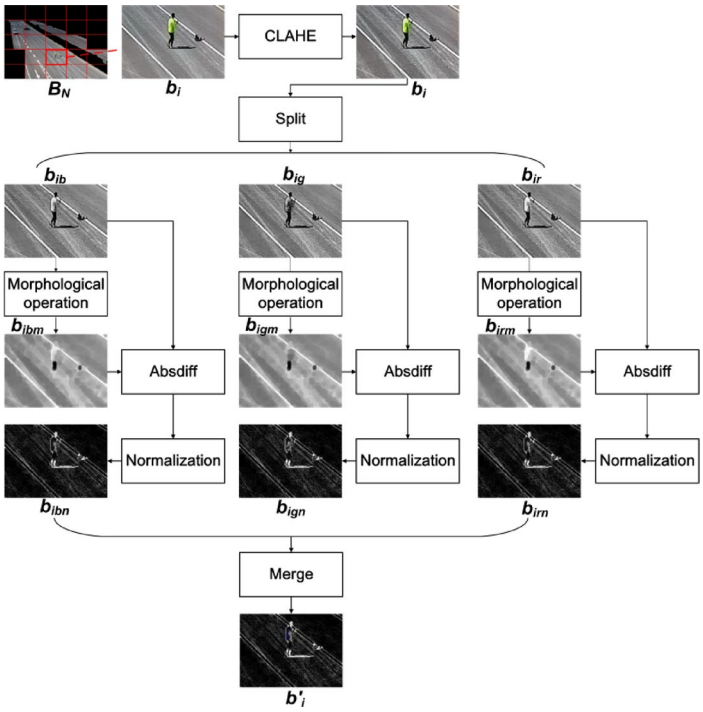


Fig. 6. The workflow of block preprocessing.

$$A' = Blur\left((A \circ B_1) \cdot B_2, B_3\right) \tag{2}$$

where A' represents the processed image after morphological operation, A represents the input binary image to be processed, B_1 is the probe used in opening ($\circ$) which is a 3×3 square structure, and B_2 is the probe used in the dilation ($\cdot$) operation, which is a 5×5 square structure. Finally, *medianblur* operation as $Blur$ is selected with 15×15 square structure as B_3.

Preprocessing methods reduce the impact of lighting changes on background modeling algorithms, which are common in highway scenes. Here foreground detection is performed on both original images and images by block preprocessing, as shown in Fig. 7, where K-Nearest Neighbors (KNN) is selected as the background modeling algorithm in scenarios with changing lighting conditions. From Fig. 7, the preprocessing step effectively suppresses the impact of lighting variations on background modeling, enabling more accurate foreground detection.

Multi-group foreground detection

To handle static foreground objects that persist throughout the sequence, we develop a multi-group foreground detection module. This module constructs a background model and produces a foreground mask with progressively higher detection frequency for stationary targets. Firstly the input sequences $\{B_1', B_2', , ,\}$ from block-based frame selection are evenly divided into *nGroup* groups $\{B_1', B_{nGroup+1}', B_{2*nGroup+1}', , ,\}$, $\{B_2', B_{nGroup+2}', B_{2*nGroup+2}', , ,\}$, $, ,$ $\{B_{nGroup-1}', B_{2*nGroup-1}', B_{3*nGroup-1}', , ,\}$. Block-based background modeling is then applied independently to each valid block in each group. Subsequently, morphological operations, as per Formula (2), with the only difference that B_3 is 5×5 square structure and Ostu[43] method for converting the grayscale image into a binary image are performed on the entire image obtained by merging foreground blocks to reduce noise. Instead of using a fixed threshold, an adaptive Gaussian model filters the target by size to further eliminate noise. The final foreground mask images $\{F_1, F_2, , ,\}$ are generated by merging the individual block masks. Figure 8 shows the total flow of multi-group foreground detection with *nRow*=*nCol*=5 and *nGroup*=2.

In addition, given that the minimum target size required by the detection system is *d*, directly using *d* to filter foreground detection targets is not ideal. Environmental factors and the sensitivity of background modeling can cause the detected foreground targets to appear smaller than *d*, as shown in Fig. 9. Setting a high threshold may result in missed detections, while a low threshold can increase false positives. However, establishing size-based distributions for abandoned objects directly is challenging due to the difficulty of collecting such samples in highway environments. Instead, we create a distribution based on size variations observed in pedestrians and vehicles in highway scenarios and use this distribution to filter litter by size.

Assuming that the real target size in the image is denoted as $S = s^* s$ and the foreground size as $S^* = s^{**} s^*$. We define the size ratio $r = s^*/s$, which is modeled as following a normal distribution. The prior distribution is denoted as $(1, \delta_0)$. According to our collected traffic videos, the sizes $\{s_1, s_2, , , s_N\}$ of traffic objects (cars, pedestrians) are obtained by object detection using *YOLOv8*[44] (see Sect. 3.5). The corresponding foreground

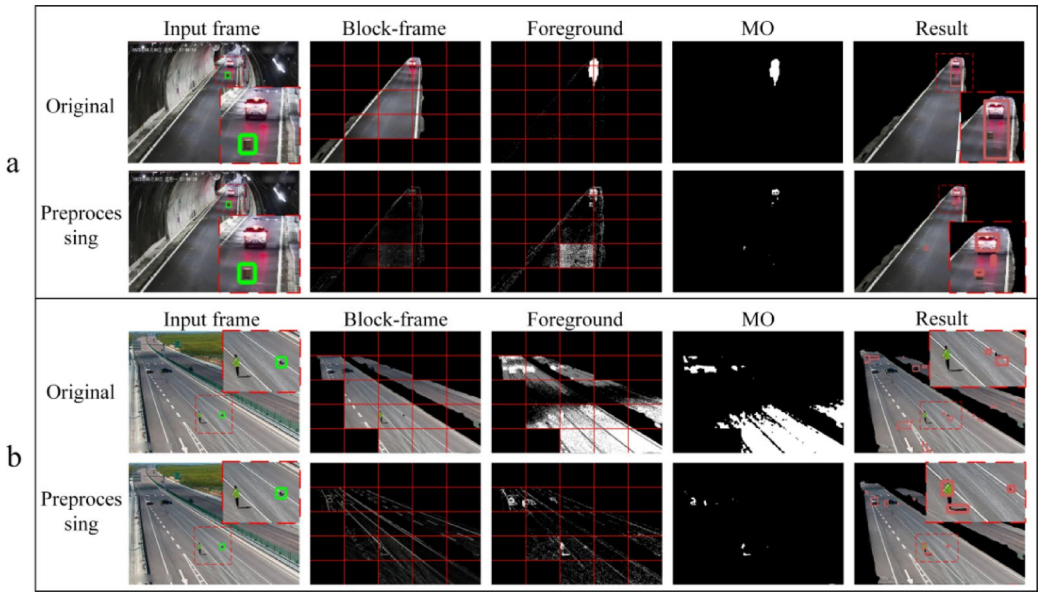


Fig. 7. Comparison of foreground detection performance between original images and images using block preprocessing. Input frame is the frame image obtained from normal video. Block-frame represents the image obtained through block-based selection which serves as the input image for subsequent foreground detection. Foreground represents the foreground image obtained using KNN method on each block of the image, MO represents the binary image by morphological operation on the foreground image, and result represents the detection result of foreground targets. The difference between 'Original' and 'Preprocessing' lies in whether the block preprocessing step is applied during block-based selection module. In case 'a', a box is discarded in tunnel by a car. Due to the influence of the taillights, the box could not be distinguished from the reflective area on the road surface without block preprocessing. Both the box and the road surface were recognized as a whole area. In case 'b', due to a change in lighting conditions, a large foreground area appears without pre-processing, making it difficult to identify people and small targets.

sizes $\{s_1^*, s_2^*, , , s_N^*\}$ are acquired via foreground detection, and size ratios are obtained $\{r_1, r_2, , , r_N\}$. Applying Bayes' theorem, we derive the following Formula (3):

$$p(r|r_1, r_2, , , r_N) = \frac{p(r) * p(r_1, r_2, , , r_N|r)}{p(r_1, r_2, , , r_N)} \tag{3}$$

Where $p(r|r_1, r_2, , , , r_N)$is the size ratio distribution, which conforms to the Gaussian distribution $N(\mu, \delta^2)$ through the conjugation characteristics. The corresponding mean μ and variance δ can be calculated by the following formula (4) and (5):

$$\mu = \frac{N}{\overline{\delta}^2} * \overline{\mu} + \frac{1}{{\delta_0}^2} / (\frac{N}{\overline{\delta}^2} + \frac{1}{{\delta_0}^2}) \tag{4}$$

$$\delta^2 = 1/(\frac{N}{\overline{\delta}^2} + \frac{1}{{\delta_0}^2}) \tag{5}$$

Where $\overline{\mu}, \overline{\delta}^2$ are the mean and variance of $\{r_1, r_2, , , r_N\}$. Given a 90% confidence interval, we can determine the corresponding range for the size ratio $\{\mu - \sigma, \mu + \sigma\}$. If the ratio of the foreground target size to the system's minimum required target size *falls* below $\mu - \sigma$ or the ratio of the foreground target size to the system's maximum required target size is greater than $\mu + \sigma$, the foreground target is filtered out. This approach enables us to establish a Gaussian distribution of size ratios based on blocks, thereby avoiding the limitations of fixed thresholds. Additionally, the mean of the ratio distribution is used to adjust the size of the detected foreground targets. If the dimensions of the foreground bounding box are $[w, h]$, the box is adjusted to $[\frac{w}{\mu}, \frac{h}{\mu}]$ in the image, ensuring that the center of the box remains unchanged.

The universal foreground extraction framework consisting of block-based frame selection and multi-group foreground detection can reduce the computation time while maintaining the effectiveness of foreground detection. Firstly, the block-based frame selection algorithm divides the image into multiple blocks, and then uses a fast block content comparison method to identify the blocks with changes. Only the blocks with changes are subjected to continuous multi-frame foreground analysis. Due to the significant spatiotemporal imbalance of highway traffic, there will always be image blocks with no content changes in the surveillance videos along the highway, thus saving time for foreground detection. Then multiple consecutive blocks will be evenly divided

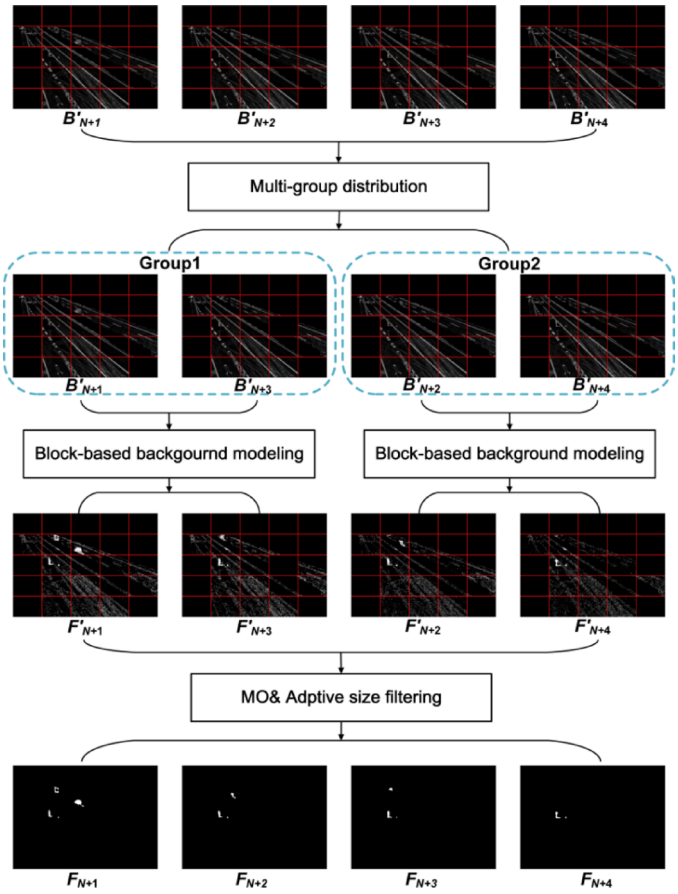


Fig. 8. The pipeline of multi-group foreground detection with $nRow = nCol = 5$ and $nGroup = 2$. MO represents morphological operation. The input frames $\{B'_{N+1}, B'_{N+2}, B'_{N+3}, B'_{N+4}, , ,\}$ is divided into two groups $\{B'_{N+1}, B'_{N+3}, , ,\}$, $\{B'_{N+2}, B'_{N+4}, , ,\}$.Then background modelling is applied on each block in each group to acquire foreground. The foreground images are obtained by merging all the blocks.as $\{F'_{N+1}, F'_{N+2}, F'_{N+3}, F'_{N+4}, , ,\}$. After morphological operation and adaptive size filtering, the final foreground sequence $\{F_{N+1}, F_{N+2}, F_{N+3}, F_{N+4}, , ,\}$ is acquired.

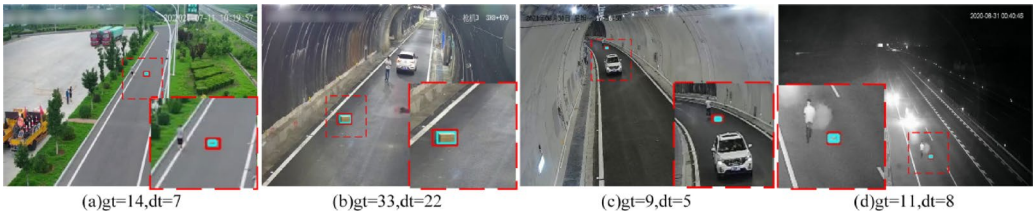


Fig. 9. Comparison of the true size of targets and detected size using KNN. 'gt' represents the true size of targets indicated in red in images, and 'dt' represents the detected size indicated in blue in images.

into different groups for foreground detection. Considering that this step is to extract foreground targets in a stationary state while the scenes on highways mainly involve vehicle motion, multi-group foreground detection is more conducive to eliminating foreground error boxes caused by slow vehicle movement and improving our accuracy, which is also reflected in Sect. 4.5. Finally, the adaptive size filtering method can effectively solve the problem of target distortion in monitoring images caused by monitoring angles, avoiding the erroneous filtering of abandoned objects caused by fixed thresholds. In Sect. 4.5, we also conducted a theoretical performance analysis of our framework UBMG, providing theoretical support for its effectiveness. According to our experiments, the overall performance is significantly improved, and the processing time is decreased by 44.3% with parameters set to $nRow = nCol = 5$ and $nGroup = 2$, compared to standard foreground detection workflows (detailed in Sect. 4.5).

Static target matching

To identify potential suspicious targets for spills, we first acquire stable prospective targets. The next step processes these detected entities to create stable candidates, ready for further verification phases. This candidate generation process is inspired by the matching strategy of associating detected and predicted targets to create trackable objects. However, our objective is to identify and maintain stable objects that remain in the same location over a specified duration, eliminating the need to predict new track locations. To achieve this, we employ both Intersection over Union (IoU) and feature-based metrics to assess object similarity using a refined static target matching algorithm, referred to as Algorithm 2, which generates stable candidate matches. Our approach integrates two principal similarity assessment techniques: *IoUMatch* and *RepNetMatch*. *IoUMatch* calculates the IoU between objects for similarity. Meanwhile, *RepNetMatch* uses features extracted from *RepNet*[45] to compute cosine similarity.

The algorithm begins by establishing three distinct sets of objects: the tentative set T, containing objects to be matched; the matched set M, containing successfully matched objects; and the candidate set C, which includes objects that have remained stable for a specified period (Line 1 in Algorithm 2). The initial matching is performed between the matched set M and the detection boxes D_k, employing *IoUMatch* to assess similarity. Unmatched detections from the detection boxes are stored in D_{remain}, while unmatched tracks from M are retained in M_{remain}. Matched targets in the matched set M increase duration time (Lines 4–6 in Algorithm 2). The unmatched detections D_{remain} need further confirmation. Then the second association is between D_{remain} and tentative set T. We use *IoUMatch* to associate, while unmatched objects from D_{remain} are kept in $D_{re-remain}$, unmatched objects from T are kept in T_{remain}. Also, we keep matched objects from D_{remain} as $D_{associate}$ and from T as $T_{associate}$ for a third match (Lines 7–11 in Algorithm 2) .

A successful IOU match does not necessarily indicate that the two targets are the same, as it may include successive objects like moving vehicles. To refine stable target identification, we apply a third match using *RepNetMatch*, which assesses similarity through feature-based metrics between $D_{associate}$ and $T_{associate}$. Targets that fail this match are retained in sets $D_{unmatch}$ and $T_{unmatch}$, respectively (see Lines 12–14 in Algorithm 2). After completing the matching process, we discard any unmatched tracks in T_{remain} which seem not to be stationary, and add unmatched tracks in $D_{re-remain}$ to a tentative set T for matching in next frame. Successfully matched objects from $T_{associate}$ are added to matched set M which are confirmed to be stationary. Subsequently, unmatched objects $T_{unmatch}$ are updated in place with $D_{unmatch}$ (Line 15–18 in Algorithm 2). Finally, stable objects persisting beyond a specified duration, t_s, are moved to candidate set C for the next validation step. Here, t_s represents the threshold for the maximum allowable time.

Input: A video sequence V, foreground detector Det
Output: Candidate set C
/* init Tentative set T,Matched set M,Candidate set C */
1 T,M,$C \leftarrow \emptyset$
2 **for** I_k **in** V **do**
3 $D_k \leftarrow$ Det(I_k); // detect foreground as D_k
/* first matching */
4 Associate M and D_k using IoUMatch
5 $D_{remain} \leftarrow$ remaining objects from D_k
6 $M_{remain} \leftarrow$ remaining object from M
/* second matching */
7 Associate T and D_{remain} using IoUMatch
8 $D_{re-remain} \leftarrow$ remaining objects from D_{remain}
9 $T_{remain} \leftarrow$ remaining objects from T
10 $T_{associate} \leftarrow T \setminus T_{remain}$
11 $D_{associate} \leftarrow D_{remain} \setminus D_{re-remain}$
/* third matching */
12 Check similarity between $T_{associate}$ and $D_{associate}$ with RepNetMatch
13 $T_{unmatch} \leftarrow$ unmatched objects from $T_{associate}$
14 $D_{unmatch} \leftarrow$ unmatched objects from $D_{associate}$
/* delete unassociated objects */
15 $T \leftarrow T \setminus T_{remain}$
/* init new objects to be matched */
16 $T \leftarrow T \cup D_{re-remain}$
/* add new candidate matched */
17 $M \leftarrow M \cup T_{associate} \setminus T_{unmatch}$
/* update unmatched objects */
18 $T \leftarrow T \setminus T_{unmatch} \cup D_{unmatch}$
19 Select objects in M with a duration longer than t_s to C
20 **end**
21 Return:C

Algorithm 2. Pseudo-code of static target matching.

Candidate verification

In this subsection, we present a methodology for tracking the trajectories of abandoned objects, identifying their transition from motion to stillness to confirm their validity. This approach effectively reduces interference from irrelevant entities, such as shadows and light reflections, through a three-step process: traffic object removal, binary noise elimination on road surfaces, and motion analysis for process verification.

Traffic object removal Highways often contain numerous traffic entities, such as vehicles and construction workers, which may trigger false alarms for abandoned objects. These entities are filtered out using object detection techniques. Here, we employ YOLOv8s[44] for object detection, leveraging its performance of approximately 170 FPS (at 640×640 resolution). The official COCO128-based training model is converted to *TensorRT* for enhanced inference speed.

Binary road noise removal To streamline the extraction of object trajectories, we implement a binary classification framework to distinguish objects from road surfaces, mitigating foreground distortions caused by artifacts and watermarks. We define two classes (road vs. non-road) and use ResNet34 as the backbone for feature extraction and classification. Image samples of size 96×96 are used to fine-tune the pre-trained model, which processes foreground regions to classify them as either part of the road or an object.

Trajectory discrimination While the preceding methods help reduce false positives, they are not entirely sufficient. Missed detections may occur due to limitations in object detection, such as distant traffic entities that are difficult to detect. Additionally, lane markings or signs may be misclassified during binary classification. Such errors differ in trajectory from abandoned objects. Stationary objects, like vehicles stopped illegally or lane markings, remain in the same position for extended periods, whereas abandoned objects typically exhibit a trajectory pattern of moving to rest. Significant progress has been made in transformer-based tracking algorithms, including ReTrackVLM[46] that integrates multimodal embedding from a visual language model (VLM) with a zero-shot re-identification (ReID) module to enhance tracking accuracy and robustness. To further refine detection, we use a lightweight tracking algorithm based on transformer combined with fast SVM discrimination for trajectory analysis due to hardware limitations and the need for multiple video analyses in highway scenes. We propose a trajectory discrimination module that utilizes target tracking to capture the trajectory, extracts trajectory intersection features, and employs an SVM model for classification to effectively filter out non-abandoned objects, as presented in Fig. 10.

To capture the target's trajectory comprehensively and minimize tracking errors due to occlusion, we select three distinct time periods for trajectory extraction. In the first stage, when a change is detected within a block, the block-based frame extraction algorithm records the first frame where the *BlockChange* function returns True as the beginning of the block-change phase. We then select m frames in reverse order to construct the tracking image sequence for this phase. In the second stage, the frame initially detected by foreground detection is designated as the start of the object-track phase, from which m frames are again selected in reverse order. In the third stage, the current frame marks the beginning of the object-station phase, and m frames are similarly chosen in reverse order. These three image sequences are combined to create a comprehensive sequence for tracking

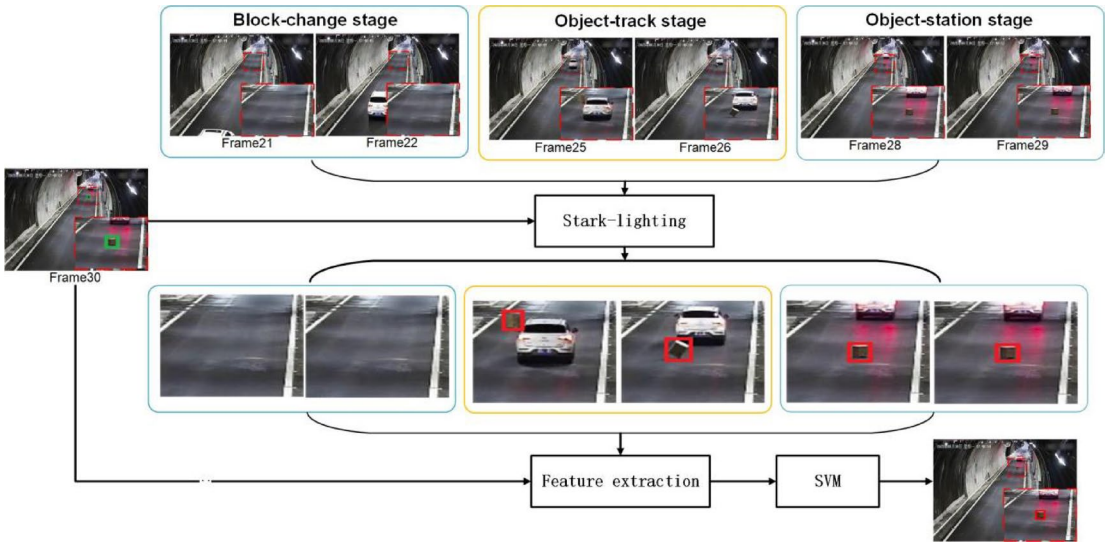


Fig. 10. The flow of trajectory discrimination. The video is extracted at 1fps for processing. Trajectory discrimination is performed on the box object in Frame 30. The corresponding block changes in Frame 22, and the object is detected as foreground in Frame 26. Then it is identified as a stationary foreground target by static object matching in Frame 30. After obtaining the trajectory of the target through stark-lighting, positional features are extracted for discrimination by SVM.

and are passed into the next stage to derive the target's trajectory. Examples of the three stages across various scenarios are illustrated in Fig. 11. By sampling images from these time periods, we can obtain a complete trajectory of the abandoned object from its motion to its stationary state. In scenarios a, b, and c, which involve litter objects, the trajectory information from movement to stillness is captured across the three stages. Scenarios d and e, which contain error targets, allow for tracking of stationary trajectories, helping to distinguish these from true abandoned objects.

To determine the trajectory of an identified object candidate, we employ the STARK-Lightning tracker[47], which is built on an advanced tracking architecture that integrates an encoder-decoder transformer. This tracker achieves performance six times greater than Siam R-CNN, delivering real-time speeds of 200–300 FPS on one RTX TITAN GPU. We designate the target in the current frame as a template and proceed through the image sequence to track its trajectory. The presence of the tracked target is evaluated based on the tracking confidence; if this confidence falls below a specified threshold T_{track}, the position for that frame is marked as empty.

In the feature extraction step, we integrate *GIOU*[48] and *IOU* methods to capture features of the target's trajectory. If box *A* indicates the final position in the discrimination frame, box *B* represents the target's position derived from trajectory tracking, and *C* is the area where they intersect. Trajectory features are calculated using *GIOU(A, B)*, *IOU(A, B)*, *IOU(A, C)*, and *IOU(B, C)*. If the target position is empty, GIOU is set to -1. Finally, the feature values with a dimension 12**m* are obtained as input data for SVM model training and discrimination, where *m* is the number of frames tracked in each stage.

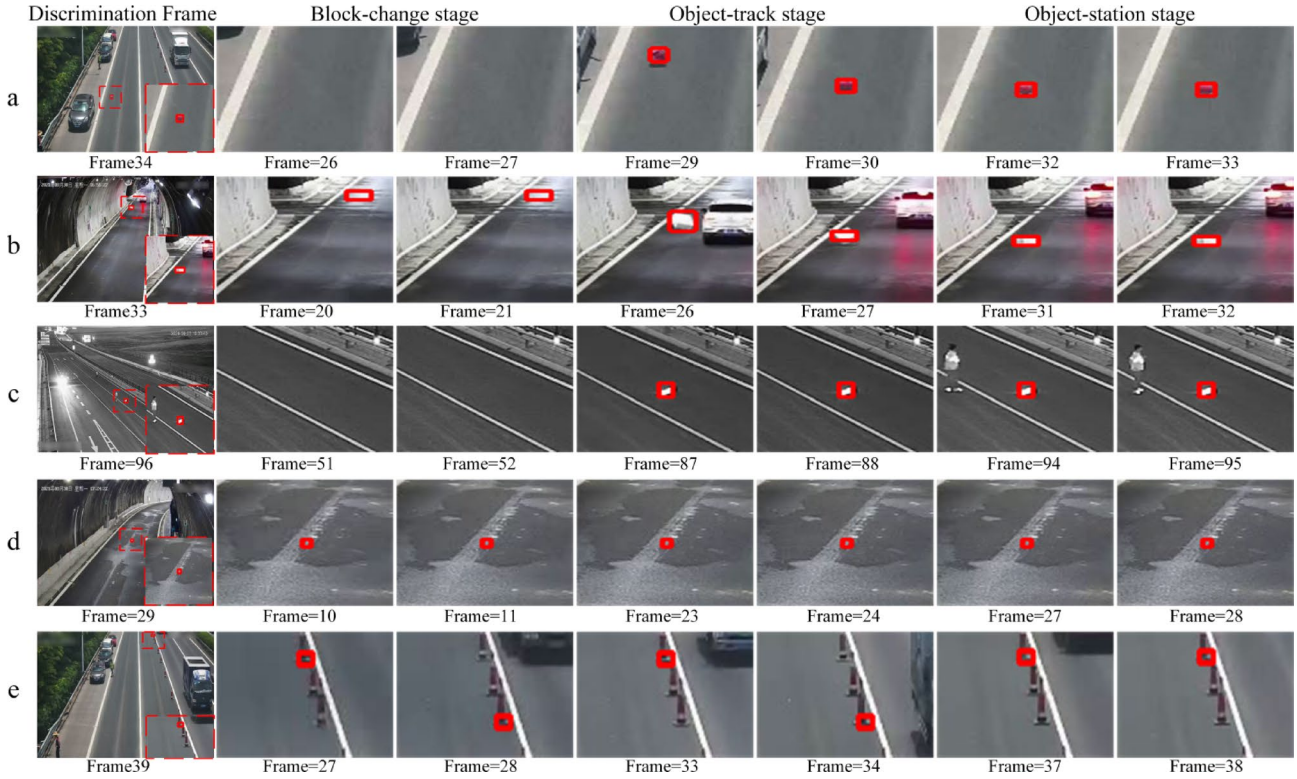


Fig. 11. The location of the detection target on different stages. Videos are extracted as 1fps for processing. Discrimination Frame indicates the current frame to be processed by Trajectory discrimination. The dotted box indicates the area tracked with STARK-Lightning[47]. Each of the next two columns corresponds to the three stages. The frame number below the image indicates the frame number in the source video. In scene 'a,' a construction hat is discarded on the road, and trajectory discrimination is used at Frame 34. At Frame 28, there is a content change in the corresponding block with no target to be tracked in the block change stage; At Frame 31, the target was detected as foreground, and the object track stage obtained the trajectory of the target's movement. And the object-station stage obtains the continuous stationary trajectory of the target. In scenario 'b,' the block change stage obtained incorrect target positions, but these positions do not match the final target position at Frame 33. In scene 'c,' a construction cone was discarded at night. Due to the influence of vehicle lighting at night, the target cannot be timely detected as foreground, so the object-track stage did not capture the trajectory of the target's movement. However, the corresponding block has been recorded as changed at Frame 53, then the block change stage could provide evidence of the target's movement. In scenario 'd', the water stains reflected were mistakenly identified as suspicious targets, and the target was detected stably in the same position in all three states. In scenario 'e', due to equipment shaking, the lower part of the construction cone was detected as foreground, and the target was correctly detected in most frames in the three stages.

Discussion

In this part, we analyze the distinctions between the UBMG approach we propose and other methods for abandoned object detection, such as Majchrowska[49], Amrutham[50], Luna[1], Nyarko[51], Giancarlo[30] and Lis[52].

1) difference from Majchrowska[49] and Amrutham[50]: Both the EfficientDet model in[49] and the YOLO based approach[50] belong to strongly supervised object detection frameworks, which performance heavily relies on large-scale annotated datasets. However, highway abandoned object datasets are extremely difficult to collect, resulting in poor generalization of such methods to unseen object categories. In contrast, UBMG innovatively adopts an unsupervised static object detection logic, which requires no labels to extract candidate objects, and further integrates the motion characteristics of debris to effectively filter and recognize them. The proposed UBMG framework not only overcomes the dependence on annotated samples but also ensures stronger adaptability and practicality when dealing with diverse debris in open highway environments.

2) difference from Luna[1] and Nyarko[51]: Both the candidate region generation algorithm in[1] and the dual-background model in[51] rely on pixel-level differences across the entire image for foreground segmentation. This full-frame background modeling results in high computational costs and limited adaptability to complex scenarios. In contrast, UBMG introduces several innovations: it reduces the computational burden of foreground detection through a block-level parallel strategy, and it enhances robustness under rapid illumination changes by combining illumination-adaptive preprocessing (i.e., CLAHE) with multi-group foreground modeling, achieving more efficient and adaptive foreground extraction while maintaining reliable detection performance..

3) difference from Giancarlo[30] and Lis[52]: The anomaly detection method in[30] and the model based on perceptual difference maps in[52] both identify anomalies through computationally intensive pixel-level feature modeling. Such approaches are associated with high complexity and encounter limited generalization in highway scenarios, where camera installation heights and angles vary significantly. In contrast, UBMG emphasizes lightweight design and practical applicability through foreground extraction combined with motion-trajectory discrimination, enabling efficient debris detection without complex pixel-level computations. Overall, UBMG not only reduces computational complexity but also enhances robustness and adaptability in diverse highway environments, making it more suitable for real-world deployment.

The HAO dataset and experiments

Video-based highway abandoned object dataset

Currently, as far as we know, no publicly available dataset focuses specifically on abandoned object detection in highway scenes. To address this gap and provide a solid foundation for evaluating our proposed algorithm, we introduce the video-based dataset of highway abandoned objects (HAO). HAO consists of 103 short video sequences captured by highway surveillance cameras, each carefully selected to include at least one abandoned object. The sequences are drawn from an extensive library of surveillance footage, ensuring diversity in object types and environmental conditions.

The videos come in various resolutions, including 720×480 and 704×576, with a frame rate of 25 fps. The dataset covers a wide range of abandoned object sizes, from as small as 10×10 pixels to as large as 50×50 pixels, featuring diverse objects such as boxes, tires, traffic cones, bottles, bags, and paper. Recognizing the critical role of environmental conditions in detection accuracy, HAO is categorized into three distinct scenarios: pavement-daytime, pavement-nighttime, and tunnel. This classification reflects the unique challenges of different lighting conditions, such as illumination changes, video jitter, occlusion, and other realistic factors that may affect detection algorithm performance. Detailed descriptions of sample sequences are provided in Fig. 12; Table 1. The details of the dataset used to train the binary noise removal model for non-road objects and roads are described in Table 2; Fig. 13; the data was collected from the Internet and surveillance videos. Training data for the non-road objects category includes boxes, tires, road cones, bottles, bags, paper, etc., mainly from the internet; meanwhile, data for road categories includes lane markings, water streaks, reflective pavement, road shadows, etc., manually cropped from the surveillance videos in real highway scenes.

Implementation details

The experiments were conducted under Python 3.8 and PyTorch 1.12.0 on an Ubuntu 22.04 LTS system, equipped with a Quadro RTX 5000 GPU and an Intel Xeon Silver 4210 CPU (2.20 GHz). All videos are preprocessed to 1fps before conducting experiments. 30% of the data samples containing multiple scenarios are selected for SVM training in trajectory discrimination and the remaining samples are used for testing. The area size range of the targets to be detected is required to be within [10*10,50*50].

In UBMG, *nRow* and *nCol* are both chosen as 5 while nGroup is set to 2 and *nExtracted* is set to 4. KNN-based background model algorithm is implemented in OpenCV-Python library with version 4.8.1. In the phase of static target matching, *IoUMatch* applies a similarity threshold of 0.3. Meanwhile, *RepNetMatch* computes cosine similarity with a threshold set at 0.15. In the phase of candidate verification, the confidence level of YOLOv8 detection is 0.3 and the confidence T_{track} of Stark-Lightning is set to 0.15. The number of frames tracked in each stage is selected 4. SVM training details are described in Sect. 4.6.

Results on HAO dataset

Based on our HAO dataset, the experiment results are shown in Table 3, while Fig. 15 provides an in-depth visual representation of the detection performance of the UBMG under review across various intricate scenarios. To further demonstrate the effectiveness of the UBMG, we conducted comparative experiments with other methods, including refs[1,30,49–52]. And Fig. 14 shows a precision-recall scatter plot across scenarios. Specifically, in Fig. 15 our block-based extraction algorithm can identify blocks that do not require foreground detection in all three scenarios, even if there are traffic targets in motion in all three scenarios, which can significantly reduce computational cost. In 'Daytime (a)', the left lane line areas are incorrectly detected as static foreground, but they

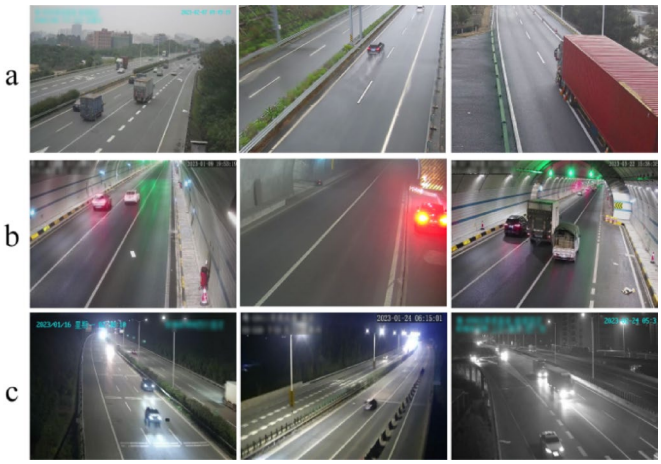


Fig. 12. Examples in HAO dataset. (**a**) shows the example of daytime scene of the road with water splashes and occlusions by vehicles; (**b**) shows the tunnel scenes including light changes and water spots; (**c**) shows the night scene of the road.

Scenario	Sequence
Overall	103
Pavement-daytime	31
Pavement-nighttime	20
Tunnel	52

Table 1. Highway abandoned object video Dataset.

Classes	Train	Validation	Test
Non-road	4941	582	246
Road	5206	504	246

Table 2. Our dataset of non-road and road.

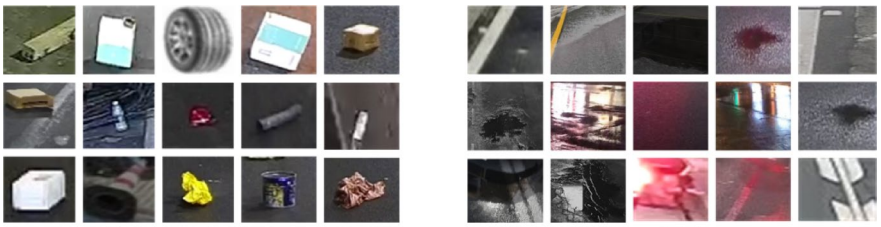


Fig. 13. Examples of roads and non-road objects. (**a**) shows examples of non-road objects and (**b**) shows road objects.

were effectively disregarded through trajectory discrimination owing to the constant, stationary nature of the lane throughout the duration of the observation. In 'tunnel (b)', although the tire was in the ground reflection area caused by the taillights, it was still well detected as foreground due to the block preprocessing algorithm. In 'nighttime (c)', the crushed construction cone was independently identified continuously for a long time through our multi-group foreground detection mechanism. The average fps is 21.

We compared the anomaly detection method proposed by Giancarlo[30], which uses the segmentation and synthesis modules trained on Cityscapes for testing. We also followed Luna[1]'s approach to legacy detection by first generating static candidate regions using MHI, and then removing vehicles and pedestrians via YOLO for abandoned object detection. The target detection model EfficientDet proposed by Majchrowska[49] was tested. We also used Amrutha's[50] method to detect litter from vehicles with YOLO. Nyarko[51] used dual temporal buffer

Method	Pavement-daytime				Pavement-nighttime				Tunnel				Average			
	P(%)	*R*(%)	f1	fps	*P*(%)	*R*(%)	f1	fps	*P*(%)	*R*(%)	f1	fps	*P*(%)	*R*(%)	f1	fps
[1](2018)	43.86	47.70	0.46	15	38.32	45.89	0.42	17	51.71	49.60	0.51	17	47.54	48.34	0.48	16
[30](2021)	38.26	47.81	0.43	8	30.12	40.32	0.34	8	43.64	72.00	0.54	7	37.33	46.13	0.41	8
[49](2022)	57.44	30.38	0.40	16	49.34	33.40	0.39	18	79.56	45.02	0.58	19	68.41	38.16	0.49	18
[50](2022)	67.72	40.54	0.51	19	56.41	42.69	0.49	19	83.56	57.12	0.68	22	73.56	46.45	0.57	20
[51](2023)	54.53	48.43	0.51	26	47.65	43.38	0.45	24	50.92	58.48	0.54	22	51.27	52.23	0.52	25
[52](2023)	33.45	42.67	0.38	6	35.67	42.45	0.38	6	39.34	60.56	0.48	5	36.58	50.34	0.42	6
Ours	94.72	92.54	0.94	21	85.32	80.54	0.83	19	91.46	87.12	0.89	22	92.93	85.27	0.89	21

Table 3. Comparison of different methods on abandoned videos.

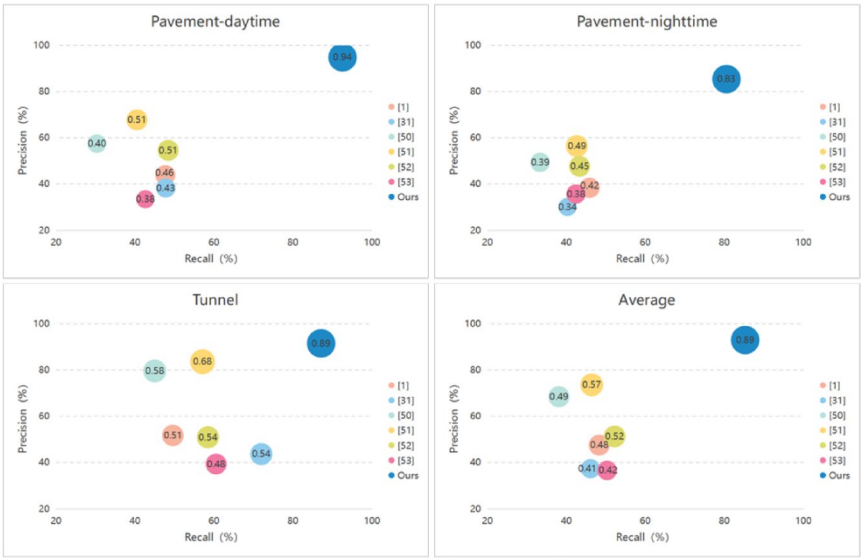


Fig. 14. Precision-recall scatter plot across scenarios. The x-axis denotes recall (%) and the y-axis denotes precision (%). Marker size is proportional to the F1-score of each method, where larger markers correspond to higher F1-scores.

differencing for independent background modeling and static foreground segmentation, combined with a single-shot deep convolutional neural network detector to detect and validate abandoned objects. Lis[52] computed a per-pixel perspective scale map to inject size consistent synthetic objects during training and to guide obstacle detection via perspective aware decoding.

Figure 16 shows the detection results of comparative methods and our algorithm on video sequences (a), (b), and (c). Giancarlo[30] successfully detects the cardboard in video sequence (b), but fails to detect small objects shown in videos (a) and (c), leading to numerous false detections because of poor generalization in segmentation and synthesis based on a limited training dataset for highway scenes. Luna[1] fails to detect objects in all three video sequences, owing to its poor performance for small object detection. Majchrowska[49] successfully detects objects in videos (b) and (c) but fails in (a) due to high sensitivity to illumination changes in highway scenes. Amrutha[50], using YOLO, achieves similar effects in nighttime and tunnel scenarios. Nyarko[51] fails to detect the paper in scene (b) and generates many false alarms due to changes in nighttime lighting. Lis[52] fails to detect litter because of inconsistent monitoring angles, which is different from scenarios for autonomous driving in Fig. 16 (a)(c). Compared with other algorithms, our method first uses block-based frame selection and multi-group foreground detection modules to better handle foreground false alarms caused by lighting changes, and then uses a static target matching module to extract the foreground in a stationary state as a candidate target. Finally, a subsequent candidate verification module is used to exclude candidate targets that do not match the motion characteristics of the abandoned objects. Our method can detect the litter well in all the videos (a), (b), (c) without generating any false alarms. In general, the proposed UBMG outperforms the compared methods and can cope with complex environments on highways.

Results on ABODA dataset

In addition to abandoned object detection on highways, we also evaluate the UBMG using the ABODA[35], which is a pedestrian abandoned object dataset including 11 sequences with a resolution of 640 × 480, representing various real-world scenarios such as crowded environments and notable changes in lighting conditions. In

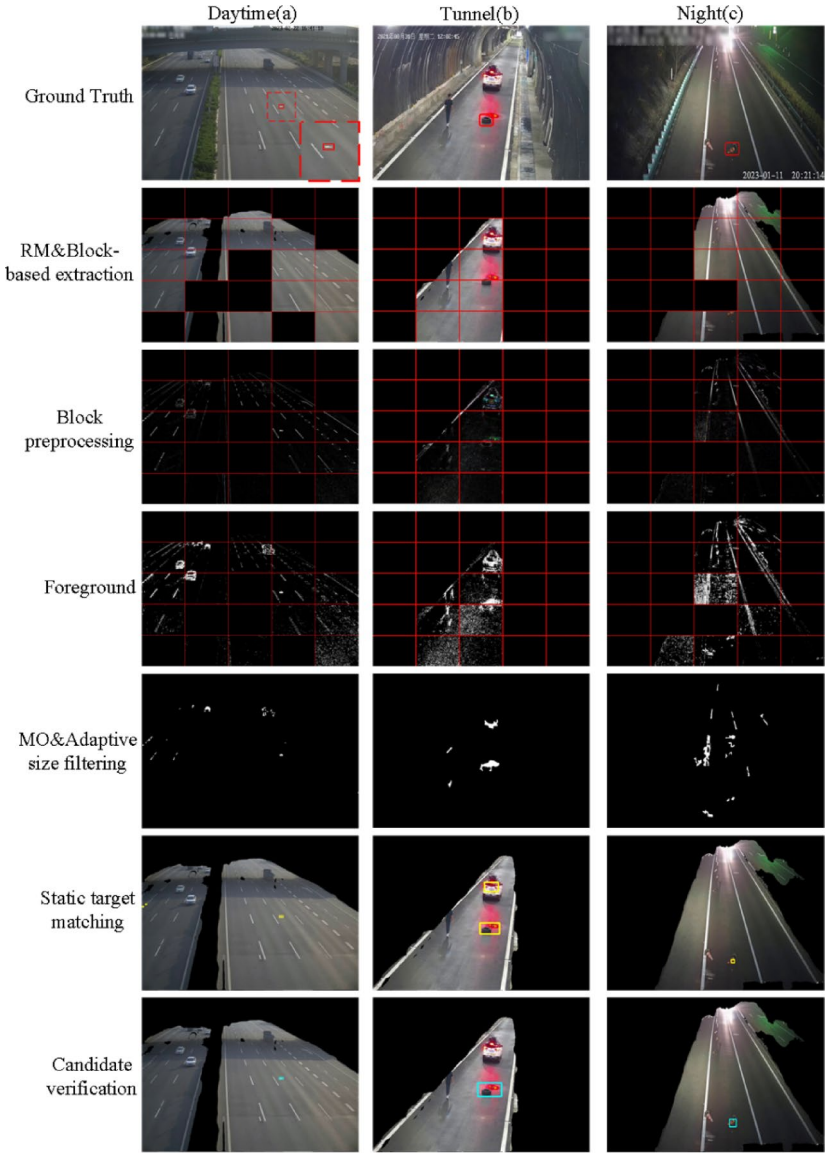


Fig. 15. The results of our algorithm on highway scenarios. 'RM' represents Road masking. MO represents morphological operation. The row of ground truth indicates the actual positions of abandoned objects. The following rows represent the results obtained from the corresponding steps of processing. In daytime (**a**) a piece of paper was discarded on the road. In tunnel (**b**) a tire was left behind the car with the taillights on. In nighttime (**c**) a construction cone was crushed and rolled by passing vehicles.

these sequences, pedestrians may leave behind objects like backpacks or luggage. For comparison, we selected 10 sequences, covering both indoor and outdoor settings. In this test, we set the detection area size range to $[10\times10,\ 100\times100]$, while keeping other configurations consistent at 1 fps extraction for our algorithm. The results are presented in Table 4, and the module outputs are illustrated in Fig. 17. At the same time, we added other algorithms related to pedestrian littering for comparison. In Table 4[14], achieves the best results[49]. merges RetinaNet, which is used to detect pedestrians, and the SSD network, which localizes hand luggage to detect an unspecified object located in the hand, and its performance may not be promising in a real environment. Our proposed algorithm shows good performance in normal scenarios.

Computational analysis of UBMG

The computational complexity of the UBMG method is analyzed to demonstrate the efficiency of our approach. We adopt a block-by-block processing method rather than parallelizing all blocks, even though our block-based framework inherently supports parallel processing. This analysis focuses on two modules: block-based frame selection and multi-group foreground detection, because combining these two modules can significantly reduce computational load while maintaining foreground detection performance. At the same time, the road masking

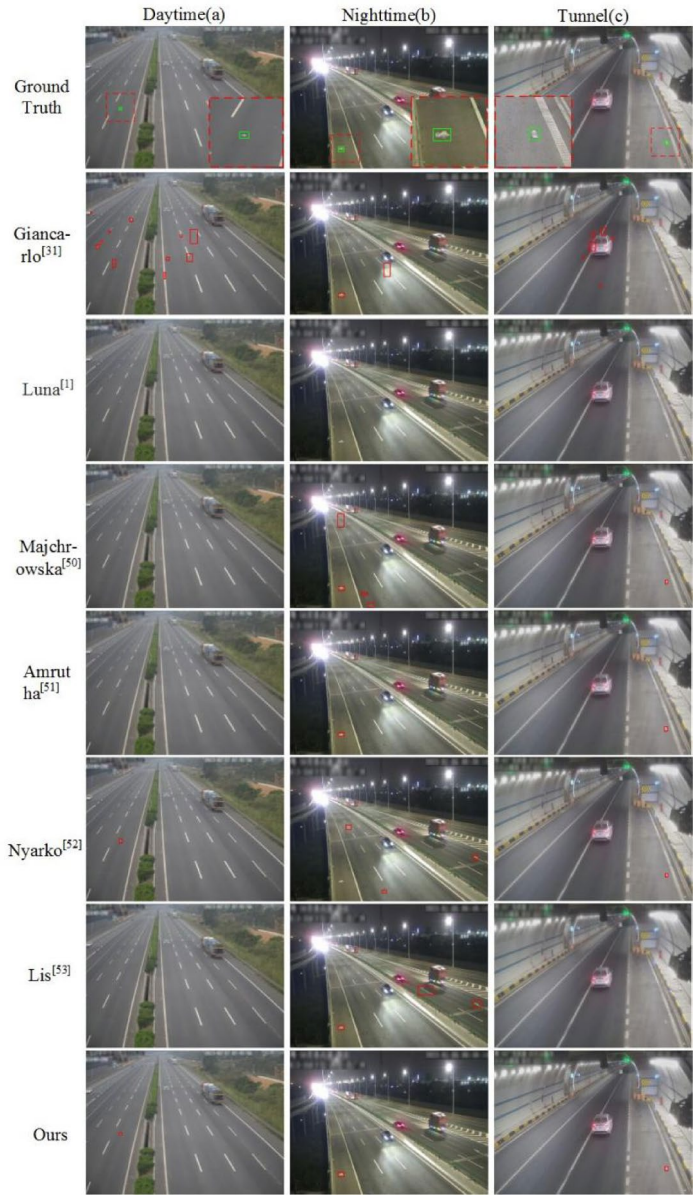


Fig. 16. Comparison of the detection results of the comparative methods and our method on video sequences (**a**), (**b**), and (**c**). In the daytime scene (**a**), a piece of paper on the road is shown. In the nighttime scene (**b**), cardboard should be detected under low-light conditions. In the tunnel scene (**c**), a plastic bottle is observed. Ground Truth represents the manually annotated bounding boxes. The detailed experimental results of Giancarlo[30], Luna[1], Majchrowska[49], Amrutha[50], Nyarko[51], Lis[52] and Ours are displayed in the figure, respectively.

part is removed in the block-based frame selection module to demonstrate the effectiveness of our architecture in general scenarios. All videos in our dataset HAO are processed only by the two modules to calculate relevant indicators.

First, we compare the time performance of the foreground detection approach with the *BlockChange* method in the block-based frame selection module on a single image block. We evaluate the time consumption of each method across various numbers of blocks, where $nCol = nRow$, as shown in Table 5, where 'pre' represents block preprocessing. Results show that the time consumption across different block quantities varies by more than an order of magnitude, underscoring the efficiency of our approach.

Impact of block number: We applied KNN (without pre), MOG2 (without pre), WeSamBE (without pre), and our pre + KNN algorithm as foreground detection methods in UBMG, respectively, to detect abandoned objects on the HAO dataset, with $nGroup = 1$, as shown in Tables 6, 7, 8, and 9. In the tables, the columns represent the following: "check/total" is the proportion of blocks that undergo the BlockChange method, "valid/total" is the proportion of blocks that do not require foreground detection because no content changes or foreground does not exist in the last frame-block, and "process_time" is the average processing time per frame. "Delay_

	Scenario	GT	30 (2021)		1 (2018)		49 (2022)		50 (2022)		51 (2023)		52 (2023)		14 (2021)		53 (2017)		35 (2015)		Ours	
			TP	FP	TP	FP	TP	FP	TP	FP	TP	FP	TP	FP	TP	FP	TP	FP	TP	FP	TP	FP
Video1	Outdoor	1	1	8	1	0	0	1	1	0	1	0	0	2	1	0	1	0	1	0	1	0
Video2	Outdoor	1	1	6	1	3	0	0	1	0	1	0	1	2	1	0	1	0	1	0	1	0
Video3	Outdoor	1	1	16	1	0	0	0	1	0	1	0	0	0	1	0	1	0	1	0	1	0
Video4	Outdoor	1	1	3	0	0	1	0	1	0	1	0	0	0	1	0	1	0	1	0	1	0
Video5	Night	1	1	6	0	2	0	0	1	0	1	1	0	2	1	0	1	0	1	0	1	1
Video6	Light switching	2	2	9	2	16	0	1	2	0	2	0	0	1	2	0	1	0	2	1	2	0
Video7	Light switching	1	1	9	1	4	0	1	1	0	1	0	0	2	1	0	1	1	1	0	1	0
Video8	Light switching	1	1	8	0	30	1	3	1	0	1	0	0	2	1	0	1	0	1	1	1	1
Video9	Indoor	1	0	9	1	0	0	2	1	0	1	0	0	0	1	0	1	0	1	0	1	0
Video10	Indoor	1	0	8	0	3	0	0	1	0	1	1	0	1	1	0	1	0	1	1	1	0
Average		1.10	0.90	8.20	0.70	5.80	0.20	0.80	1.1	0	1.1	0.2	0.1	1.3	1.1	0	1	0.1	1.1	0.3	1.1	0.2

Table 4. Comparing result on ABODA dataset sequence.

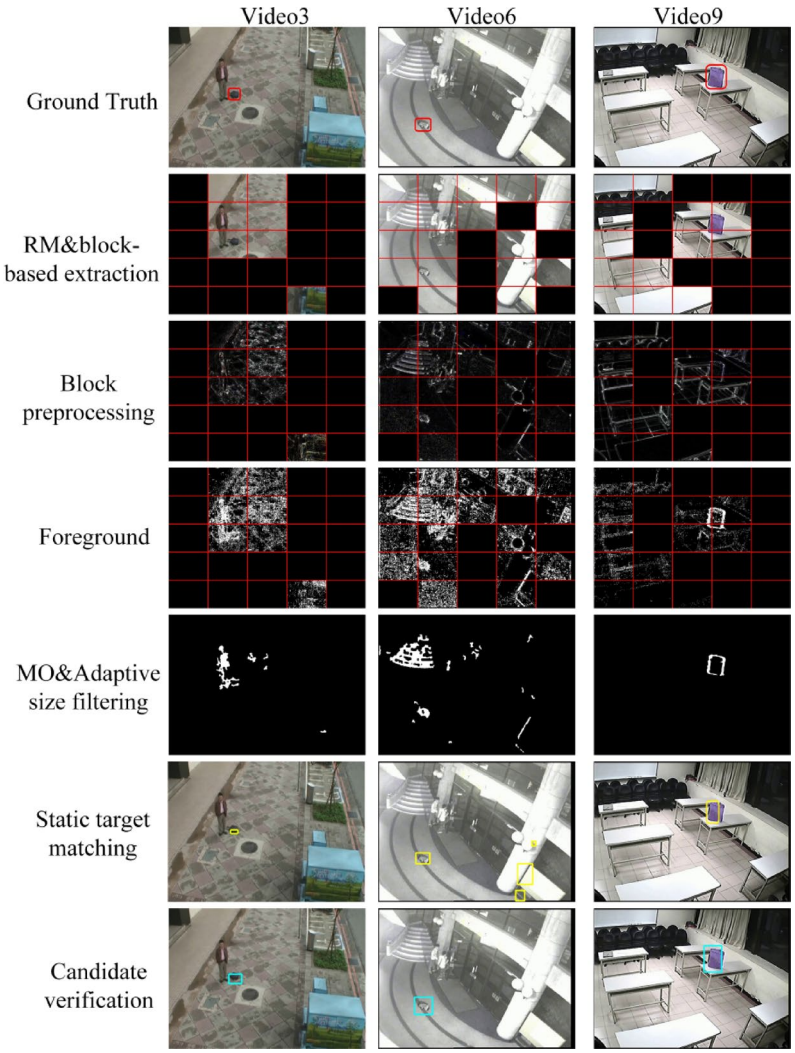


Fig. 17. The results of our algorithm on ABODA. In Video 3, a black bag was discarded beside a pedestrian; in Video 6, a bag was left in the hall while the lighting had just changed; in Video 9, a purple bag was dropped on the desk in the room.

Method	nRow = 1	nRow = 2	nRow = 3	nRow = 4	nRow = 5	nRow = 6	nRow = 7	nRow = 8
BlockChange	0.67(1)	0.15(1)	0.07(1)	0.05(1)	0.03(1)	0.03(1)	0.03(1)	0.02(1)
mog2	12.93(28.54)	2.66(17.73)	1.29(18.43)	0.76(15.2)	0.5(16.67)	0.36(12)	0.26(8.67)	0.2(10)
KNN	19.12(12.42)	5.08(33.87)	2.47(35.29)	1.48(29.6)	1(33.33)	0.72(24)	0.55(18.33)	0.43(21.5)
vibe	8.32(612.13)	2.1(14)	1.01(14.43)	0.61(12.2)	0.42(14)	0.31(10.33)	0.24(8)	0.19(9.5)
WeSamBE	610.13(910.64)	219.79(1465.27)	87.46(1249.43)	56.38(1127.6)	43.2(1440)	33.25(1108.33)	26.74(891.33)	22.61(1130.5)
pre + KNN	69.42(103.61)	18.11(120.73)	8.64(123.43)	5.18(103.6)	3.55(118.33)	2.55(85)	1.97(65.67)	1.6(80)

Table 5. Average time consumption of different methods with different nRow on one block (Unit: Milliseconds). The value in parentheses represents the time consumption ratio of the corresponding method and BlockChange method. 'pre' represents block preprocessing.

Method	nRow	Process_time(ms)	Check/total	Valid/total	Precision	Recall	f1-score	Delay_time
KNN (w/o pre)	/	19.12	/	/	2.92%	76.49%	0.06	2.73
	1	18.29	0.09	0.08	2.92%	76.49%	0.06	2.73
	2	16.21	0.34	0.31	2.92%	74.31%	0.06	3.23
	3	14.04	0.53	0.5	3.20%	76.76%	0.06	3.73
	4	13.39	0.59	0.57	3.34%	74.40%	0.06	3.83
	5	12.72	0.65	0.64	3.12%	74.58%	0.06	3.73
	6	12.42	0.69	0.67	3.21%	72.68%	0.06	4.01
	7	12.42	0.71	0.7	3.37%	75.69%	0.06	3.93
	8	12.48	0.73	0.72	3.57%	70.05%	0.07	4.73

Table 6. Metrics of KNN method with different nRow.

Method	nRow	Process_time	Check/total	Valid/total	Precision	Recall	f1-score	Delay_time
WeSamBE (w/o pre)	/	410.13	/	/	3.47%	76.21%	0.07	4
	1	388.16	0.05	0.04	3.47%	76.21%	0.07	4
	2	313.74	0.23	0.22	4.30%	74.40%	0.08	4.33
	3	274.79	0.44	0.43	4.20%	76.30%	0.08	4.18

Table 7. Metrics of WeSamBE method with different nRow.

Method	nRow	Process_time	Check/total	Valid/total	Precision	Recall	f1-score	Delay_time
MOG2 (w/o pre)	/	12.93	/	/	2.35%	58.89%	0.05	2.7
	1	11.43	0.13	0.11	2.35%	58.89%	0.05	2.7
	2	9.74	0.38	0.34	2.14%	54.00%	0.04	2.87
	3	8.48	0.55	0.52	2.26%	55.63%	0.04	2.98
	4	8.13	0.62	0.59	2.11%	52.12%	0.04	3.09
	5	7.85	0.67	0.65	2.32%	53.93%	0.04	3.79
	6	7.84	0.7	0.68	2.15%	51.32%	0.04	3.11
	7	6.91	0.73	0.71	2.19%	48.97%	0.04	3.5
	8	6.17	0.74	0.73	2.16%	46.05%	0.04	3.51

Table 8. Metrics of MOG2 method with different nRow.

time" indicates the difference between the detection time and the actual occurrence time of the abandoned object. The first row in each table shows the results of directly applying the corresponding foreground detection method without UBMG. The findings suggest that as the number of blocks increases, the values of "check/total" and "valid/total" also increase, while "process_time" decreases. However, since our approach processes blocks sequentially rather than in parallel, further reductions in block size beyond a certain threshold diminish time gains due to machine caching limitations. Precision and recall remain stable across different block numbers, though the block pre + KNN algorithm demonstrates a rise in recall as block numbers increase, indicating that smaller blocks allow the preprocessing algorithm to extract finer details, especially in detecting small objects.

Method	nRow	process_time	Check/total	Valid/total	Precision	Recall	f1-score	Delay_time
Ours (pre + KNN)	/	69.42	/	/	3.10%	81.23%	0.06	2.32
	1	66.95	0.1	0.08	3.10%	81.23%	0.06	2.33
	2	58.09	0.34	0.31	3.07%	86.38%	0.06	2.19
	3	42.75	0.52	0.49	3.19%	86.72%	0.06	2.89
	4	38.24	0.57	0.55	2.99%	85.62%	0.06	2.01
	5	34.92	0.62	0.6	2.82%	91.54%	0.05	2.71
	6	32.69	0.64	0.62	2.95%	90.82%	0.06	3.13
	7	31.56	0.66	0.64	2.07%	91.21%	0.04	3.7
	8	31.52	0.67	0.65	1.71%	89.31%	0.03	3.7

Table 9. Metrics of preprocess + KNN method with different nRow.

Method	nGroup	nRow	Process_time	Check/total	Valid/total	Precision	Recall	f1-score	Delay_time
KNN (w/o pre)	/	/	19.12	/	/	3.00%	76.00%	0.06	2.73
	1	5	12.72	0.65	0.64	3.94%	75.96%	0.07	3.69
	2	5	13.17	0.63	0.62	3.76%	77.92%	0.07	3.54
	3	5	14.08	0.6	0.58	2.10%	79.42%	0.04	4.04
	4	5	15.45	0.56	0.55	2.12%	79.42%	0.04	4.75
pre + KNN	/	/	27.22	/	/	2.00%	65.00%	0.04	2.32
	1	5	20.25	0.62	0.6	2.82%	91.54%	0.05	2.25
	2	5	22.64	0.57	0.55	2.72%	94.77%	0.05	2.57
	3	5	23.58	0.52	0.5	2.31%	95.59%	0.05	2.41
	4	5	25.25	0.47	0.46	1.91%	97.31%	0.04	3.16
MOG2 (w/o pre)	/	/	12.93	/	/	2.00%	59.00%	0.04	2.70
	1	5	7.85	0.67	0.65	3.20%	54.00%	0.06	3.79
	2	5	8.05	0.67	0.65	3.08%	61.53%	0.06	3.55
	3	5	9.06	0.67	0.65	2.54%	63.86%	0.05	3.73
	4	5	9.12	0.67	0.65	2.13%	44.15%	0.04	3.95
WeSamBE (w/o pre)	/	/	410.13	0.05	0.04	3.47%	76.21%	0.07	4.23
	1	5	292.63	0.59	0.58	4.02%	74.51%	0.08	6.36
	2	5	323.99	0.57	0.56	3.53%	78.89%	0.07	5.85
	3	5	358.62	0.55	0.52	3.28%	82.01%	0.06	5.75

Table 10. Metrics of methods with nRow = nCol = 5 and different nGroup.

Impact of group number: Table 10 presents the detection metrics for various values of *nGroup* with *nRow* fixed at 5. The results indicate that as *nGroup* increases, both the recall rate and processing time increase; however, the processing time remains significantly lower compared to scenarios without our framework. This demonstrates that implementing parallel block-based processing in practical applications optimizes processing time while maintaining effective detection performance. In our other experiments, we selected $nGroup = 2$ and $nRow = nCol = 5$ as default parameters for block-by-block processing.

Meanwhile, Table 10 also demonstrates that the proposed block-based frame selection approach and multi-group foreground detection method can significantly reduce the impact of illumination variations in different highway scenarios. For each method, the first row represents the effect of using the original foreground extraction algorithm, and the other rows represent the effects of our framework using different nGroup values. It can be concluded that our framework achieves significantly better detection results with specific nGroup values than the original foreground extraction algorithm alone. Specifically, for the pre + KNN method, the framework can achieve the best F1-score of 0.05 with nGroup = 2 and nRow = 5, which are selected as the parameters for our comparative experiments.

We revisit the framework UBMG based on block-based frame selection and multi-group foreground detection. Its core idea is to divide the image into blocks and then construct multiple groups of blocks for foreground detection. This approach does not conflict with the workflow of most current foreground detection algorithms, and it can be well combined with current pixel-wise foreground detection algorithms without incurring significant computational overhead, thereby enhancing the detection performance of the underlying algorithm.

The time efficiency of UBMG is analytically evaluated as follows. We focus on the time analysis of one block since images are processed in parallel by dividing into blocks in practical applications. t_c is the running time of *BlockChange* function, and t_e is the running time of foreground extraction algorithm such as KNN, MOG2

on one block. In our algorithm, if the block is marked as changed by *BlockChange* function, the image of the block will be subjected to *L* consecutive foreground extractions including the current block, without the need for *BlockChange* function discrimination. p is the probability of the block being marked as changed.

Function *BlockChange* executed each time can be seen as a Bernoulli experiment with the probability p of success. According to the properties of the geometric distribution, the expected number of experiments required until the first success is $1/p$. After obtaining the changed state, a total of *L* blocks, including the current block, need to be continuously processed by the foreground extraction algorithm. So the expected number of blocks in one cycle is $1/p + L - 1$, and the expected running time of one cycle is $1/p{*}t_c + L{*}t_e$. For *n* consecutive image blocks, the total running time T_u could be calculated as formula (6):

$$T_u = \frac{n}{1/p + L - 1} * (1/p * t_c + L * t_e) \tag{6}$$

Without using UBMG, the foreground extraction algorithm will be executed on total *n* consecutive image blocks and the total running time T_e is as formula (7):

$$T_e = n * t_e \tag{7}$$

the efficiency gain T_u/T_e is calculated as formula (8):

$$T_u/T_e = \frac{t_c/t_e + pL}{pL + 1 - p} \tag{8}$$

In Table 5, it is concluded that the proportion value $t_c/\ t_e$ is very small, approximately considered as 0. The efficiency gain can be further obtained as formula (9):

$$T_u/T_e \approx \frac{pL}{pL + 1 - p} \tag{9}$$

In our experiment, *L* is equal to *nExtracted* set to 4. Obviously, the probability *p* is related to the number of blocks in the image segmentation, as well as the scope and content of the monitoring. Our algorithm will clearly benefit from highway scenarios where the monitoring range is wide and there is a significant spatiotemporal imbalance in traffic flow.

Ablation study

Ablation experiments were conducted on the HAO dataset to evaluate the contribution of individual components within the algorithm, as shown in Table 11. 'BS' represents block-based frame selection. 'MG' represents multi-group foreground detection. 'STM' represents the static target matching, and 'TD' represents the trajectory discrimination modules. Collectively, these modules significantly enhance detection performance.

To validate the contribution of each individual component in the proposed algorithm, we performed a series of ablation experiments on the HAO dataset, and the results are summarized in Table 11. The experiments are organized in two stages: (1) progressive addition of modules to the baseline model (Exp. 1–4), and (2) removal of individual modules from the full model (Exp. 5–11).

The baseline model (Exp. 1), with no enhancement modules, achieves an extremely low F1-score of 0.06, indicating poor detection quality. The full model (Exp. 4), integrating all four modules, achieves the best overall performance with 92.93% precision, 85.27% recall, and an F1-score of 0.89. Specifically, the BS + MG modules we proposed play an important role in the algorithm. With the addition of BS + MG (Exp. 2), the recall rate improves to 94.77%, confirming its role in reducing the impact of illumination variations. The most severe

#	BS	MG	STM	TD	precision	recall	f1-score
1(Baseline)	✗	✗	✗	✗	2.92%	76.49%	0.06
2(+ BS + MG)	✓	✓	✗	✗	2.72%	94.77%	0.05
3(+ BS + MG + STM)	✓	✓	✓	✗	65.84%	90.64%	0.76
4(Full Model)	✓	✓	✓	✓	92.93%	85.27%	0.89
5(-STM only)	✓	✓	✗	✓	74.43%	88.32%	0.81
6(-MG only)	✓	✗	✓	✓	80.35%	78.43%	0.79
7(-BS only)	✗	✓	✓	✓	89.34%	73.49%	0.81
8(-BS-MG)	✗	✗	✓	✓	85.14%	69.41%	0.76
9(-BS-TD)	✗	✓	✓	✗	65.16%	82.94%	0.73
10(-MG-TD)	✓	✗	✓	✗	60.93%	82.30%	0.7
11(-BS-MG-TD)	✗	✗	✓	✗	63.16%	79.13%	0.7

Table 11. Ablation results on abandoned videos. Blank denotes the component is used by default, while ✗ represents the component is removed.

#	Foreground extraction	Static target extraction	precision	recall	F1-score
1	KNN	STM	90.45%	74.64%	0.82
2	WeSamBE	STM	88.56%	80.47%	0.84
3	MOG2	STM	85.81%	62.38%	0.72
4	BS + MG	DeepSort[56]	72.43%	66.75%	0.69
5	BS + MG	ByteTtack[57]	82.45%	72.86%	0.77
6	BS + MG	StrongSORT[58]	86.78%	83.43%	0.85
7	BS + MG	Deep OC-SORT [59]	80.47%	75.43%	0.78
8	BS + MG	BoostTrack++ [60]	87.34%	79.39%	0.83
9	BS + MG	STM	92.93%	85.27%	0.89

Table 12. The results of different module replacement on abandoned videos.

#	Method	Precision	Recall	F1-score
1	YOLOv8	32.46%	70.38%	0.44
2	YOLOv8 + ByteTtack	57.78%	42.47%	0.49
3	YOLOv8 + STM	76.36%	63.38%	0.69
4	YOLOv8 + TD	70.43%	65.36%	0.68
5	YOLOv8 + STM + TD	88.32%	58.73%	0.71

Table 13. Extended ablation study on YOLOv8 baseline with STM and TD modules.

performance loss of recall occurs when both BS and MG are excluded (Exp. 8), resulting in recall of only 69.41%, underscoring the complementary nature of these modules.

Overall, these ablation results demonstrate that each proposed module contributes meaningfully to the system's performance. Their combined effect is essential for robust detection in complex traffic surveillance scenarios.

In order to further verify the effectiveness of the proposed multiple modules, this paper replaces the BS + MG module and STM module with other foreground extraction algorithms and static target extraction strategies, while keeping the other modules of the algorithm unchanged, to compare the detection performance.

The selected foreground extraction algorithms include KNN, WeSamBE, and MOG2, which directly use the size range of[10,49] to filter suspicious targets. The static target extraction strategy directly uses tracking algorithms for foreground target tracking. When the IOU values of the target for 4 consecutive frames are all greater than 0.3, it is considered that the target is in a static state. The comparative tracking algorithms include DeepSort[54], ByteTtack[55], StrongSORT[56], Deep OC-SORT[57], BoostTrack++[58]. These algorithms encompass object tracking algorithms that have demonstrated strong performance in recent years[56]. The experimental results are shown in Table 12.

From Table 12, it is concluded that BS + MG module for foreground extraction demonstrates significant improvement in indicators compared to common foreground extraction algorithms. At the same time, STM module is specifically designed for stationary target extraction, which has a significant improvement in effectiveness compared to directly using tracking algorithms for stationary target extraction. Our method combining BS + MG module and STM module performs the best in terms of relevant indicators.

To further evaluate the effectiveness and generalization of the proposed modular design, we conducted additional experiments by embedding the modules into an advanced object detection framework. Specifically, YOLOv8[44] was employed as the baseline detector to replace the original foreground extraction component. All experiments were conducted under the same highway litter detection setting, and redundant alarms were removed during evaluation to ensure fair comparisons. As shown in Table 13, introducing STM module leads to higher precision by effectively filtering false positives from stationary background regions, and TD module improves precision by capturing temporal variations more robustly. The combination of STM and TD achieves the best performance in terms of F1-score.

Effects of adaptive size filtering: Using our HAO dataset, we employed YOLOv8 to detect vehicle and pedestrian targets, integrating our foreground detection method to isolate relevant foreground elements. This approach generated matched sample pairs with an IoU greater than 0.2. Given the target size range of[10,49] annotated in our dataset, we filtered a total of 38,838 pairs that met this criterion. A prior distribution for the size ratio was established as N(1, 0.1), from which we derived the size ratio distribution for traffic-related targets. Similarly, we applied this process to abandoned objects labeled in our dataset, yielding 7,582 matched pairs. The same statistical analysis was conducted to determine the size ratio distribution for these abandoned objects, and the resulting distributions are shown in Fig. 18. The results highlight a strong similarity between the size ratio distribution derived from traffic targets and the annotated abandoned objects. However, while the annotated data is limited in quantity and scene-specific, our object detection method remains robust, unaffected by sample size limitations or specific real-world settings, making it adaptable across diverse environments. By using a 90% confidence interval based on the traffic target distribution, we determined the optimal foreground

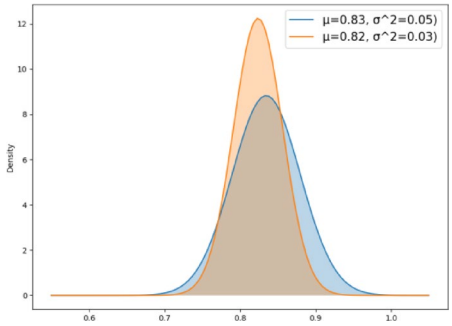


Fig. 18. Gaussian distribution of size ratio. The pink curve represents the distribution of size ratios based on traffic targets, while the blue curve represents the distribution of size ratios based on annotated targets.

Threshold	[10,50]	[6,52] (ours)
Precision	95.48%	92.93%
Recall	68.70%	85.27%

Table 14. The indicators for foreground filtering using different sizes.

	GIoU (A, B)		IoU (A, B)		IoU (A, B), IoU (A, C), IoU (B, C)		GIoU (A, B), IoU (A, B), IoU (A, C), IoU (B, C)	
	label = 0	label = 1	label = 0	label = 1	label = 0	label = 1	label = 0	label = 1
Precision	0.92	0.88	0.88	0.85	0.92	0.88	0.96	0.9
Recall	0.94	0.85	0.95	0.82	0.95	0.9	0.98	0.92
f1	0.93	0.86	0.91	0.83	0.93	0.89	0.97	0.91

Table 15. Indicators obtained by SVM training with different trajectory features.

filtering size range to be [6.22, 52.35]. In our procedure, size filtering is applied using this calculated range[6,52], in conjunction with the system's required detection range of[10,49]. Performance indicators are presented in Table 14, demonstrating that our adaptive size filtering method enhances detection metrics and offers broad applicability across different scenes.

Effects of the trajectory discrimination module: For the trajectory discrimination process, we collected target trajectories from 31 training videos in our dataset. This yielded a total of 5,516 trajectory records, including 334 true trajectories of annotated objects. To increase the sample size, we did not remove duplicate trajectories from repeated alarms. We employed the SVC method from the *sklearn* library to build an SVM model, using the polynomial kernel function and a penalty coefficient of 5. The data was split into training and testing sets with a 0.5 ratio. Experiments were conducted on various combinations of trajectory features, as shown in Table 15. Besides, it can be concluded that the three-stage trajectory analysis effectively eliminates erroneous targets, achieving accuracy and recall rates above 90% for label = 0 across various trajectory features. Incorporating GIoU features further enhances detection performance, with the optimal indicators achieved using a combination of GIoU (A, B), IoU (A, B), IoU (A, C), and IoU (B, C) features. Consequently, these four area-based features are selected for SVM training with an input dimension of 48. In our procedure, we use TRACLUS[59] and various strategy methods to replace our trajectory discrimination method for comparative evaluation, as shown in Table 16. The TRACLUS algorithm segments trajectories into line segments, clustering these to identify key sub-trajectories. Sub-trajectories containing three or fewer trajectories are flagged as abandoned objects. Strategy 1 assesses target movement frequency, where IoU < 0.3 indicates movement. If the movement ratio exceeds 0.4, the target is classified as being in motion and considered an abandoned object. Strategy 2 divides each trajectory into two equal segments and applies Strategy 1 to evaluate movement in each. If the first segment is in motion while the second is stationary, the target is identified as a spilled object. Table 15 presents the results, showing that our multi-stage SVM trajectory discrimination method achieves the highest metrics.

Failed cases

Apart from the significant performance improvement, our method may encounter recognition errors in some highly challenging situations, which reflects the limitations of the current model in practical applications. It mainly includes the following aspects:

Method	TRACLUS [62]	Strategy1	Strategy2	Ours
Precision	40.48%	65.93%	78.93%	92.93%
Recall	38.70%	78.39%	62.37%	85.27%

Table 16. Indicators from different trajectory discrimination strategies.

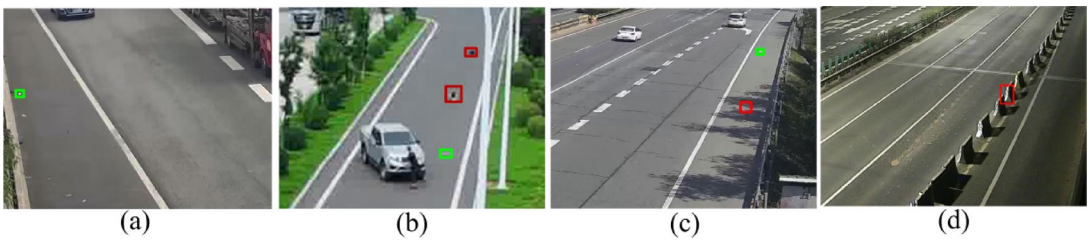


Fig. 19. Failure cases of our algorithm. In (**a**), the paper continued to roll due to a missdetection in the green box. In (**b**), a box was erroneously filtered for similarity to the road in the green box. In (**c**), the sudden appearance of tree shadows caused a false alarm in the red box. In (**d**), the isolation barrier caused false alarms in the red box because the region resembled nearby facilities, leading to trajectory tracking error.

a) *Inability to detect rolling targets*: Since the goal of our algorithm is to detect stationary objects abandoned on the road, continuously rolling targets such as paper blown by the wind will be filtered by our static target matching module, as shown in Fig.19a.
b) *Sensitivity to small or distant objects*: Small-scale or far-distance objects often lack salient features and are easily overwhelmed by the background. Additionally, small objects with visual similarity to the road surface may be mistakenly filtered out during binary road noise removal, resulting in false negatives, as shown in Fig. 19b.
c) *Detection errors caused by background dynamics*: Under the complex lighting scene of highways, there are instances of trajectory matching errors in the shadows of trees and local road facilities, resulting in false alarms as shown in Fig. 19c, d. These issues require deeper networks to extract texture features of targets and longer time to track target trajectories, but this also requires more computational resources and processing time.

Generalization challenges and model limitations

Although UBMG achieves significant performance improvement compared to existing methods, its generalization ability remains challenged by several factors inherent to complex highway environments. These limitations are complementary to the failure cases described in Section X and provide a broader perspective on the model's applicability:

a) *Cross-scenario generalization*: UBMG demonstrates reduced adaptability when deployed across scenarios with markedly different traffic densities, camera perspectives, or road structures. For example, modules optimized for highway traffic may show decreased precision in urban expressways with significantly higher flow density or in rural roads with irregular boundaries. This highlights the difficulty of transferring the model across diverse deployment conditions without adaptation.
b) *Extreme environmental conditions*: Under challenging conditions such as backlighting, glare, heavy rain, or snow, foreground extraction and trajectory verification are prone to instability. While UBMG incorporates CLAHE enhancement and multi-group background modeling, these strategies cannot fully mitigate the visual disturbances caused by severe illumination changes or weather artifacts.
c) *Object-specific detection blind spots*: Very small debris items and continuously rolling objects remain problematic for detection. The static-object assumption of UBMG leads to the exclusion of rolling debris, while the minimal block-size constraint and road-noise suppression can filter out extremely small objects. These issues point to the need for refined strategies tailored to different debris types.

Conclusion

To address the challenges of abandoned object detection on highways, this study presents a universal foreground detection framework, UBMG, along with a dedicated method to mitigate false alarms caused by highway lighting variations. Additionally, we introduce the HAO dataset, a comprehensive collection of fully annotated video sequences across three distinct highway scenarios. This dataset is designed to facilitate robust evaluation of various detection approaches. The proposed method significantly improves key performance metrics, including precision and recall, outperforming existing methods on the HAO dataset and showing good performance on the public ABODA dataset-even though the ABODA dataset is not designed for scenarios where pedestrians discard their luggage. Future work will focus on addressing other challenging environments, such as scenes with dense pedestrian activity, and researching new hybrid frameworks (e.g., combining deep learning for candidate generation and UBMG for verification).

Data availability

The highway video datasets and code used in this study are available from the corresponding author upon reasonable request. Due to privacy and licensing restrictions associated with vehicle and pedestrian imagery, raw video data cannot be shared publicly. Access to the full dataset may be granted under a data use agreement with the data provider or through institutional approval.

Received: 18 April 2025; Accepted: 15 September 2025
Published online: 27 October 2025

References

1. Luna, E., San Miguel, J. C., Ortego, D. & Martínez, J. M. Abandoned object detection in video-surveillance: survey and comparison. *Sensors* **18**, 4290 (2018).
2. Amin, F., Mondal, A. & Mathew, J. A large dataset with a new framework for abandoned object detection in complex scenarios. *IEEE MultiMedia*. **28**, 75–87 (2021).
3. Shyam, D., Kot, A. & Athalye, C. Abandoned object detection using pixel-based finite state machine and single shot multibox detector. In: *Proceedings of the IEEE International Conference on Multimedia and Expo (ICME)*, (IEEE, 2018) (1-6).
4. Shi, H. & Liu, C. A new cast shadow detection method for traffic surveillance video analysis using color and statistical modeling. *Image Vis. Comput.* **94**, 103863 (2020).
5. Ilias, D., Mezouar, E. & Taleb, M. C. An edge-based method for effective abandoned luggage detection in complex surveillance videos. *Comput. Vis. Image Underst.* **158**, 141–151 (2017).
6. Liu, W. et al. General-purpose abandoned object detection method without background modeling. In: *Proceedings of the IEEE International Conference on Imaging Systems and Techniques (IST)* (IEEE, 2021).
7. Kim, D. et al. HLDNet: abandoned object detection using hand luggage detection network. *IEEE Consum. Electron. Mag.* **11**, 45–56 (2021).
8. San Miguel Avedillo, J. C. & Martínez Sánchez, J. M. Robust unattended and stolen object detection by fusing simple algorithms. In: *Proceedings of the IEEE Conference* (2008).
9. Wang, Z. et al. Image quality assessment: from error visibility to structural similarity. *IEEE Trans. Image Process.* **13**, 600–612 (2004).
10. Wojke, N. & Bewley, A. Deep cosine metric learning for person re-identification. In:*Proceedings of the IEEE Winter Conference on Applications of Computer Vision (WACV)* (IEEE, 2018), (748–756).
11. Pinto, L., Andriolo, U. & Gonçalves, G. Detecting stranded macro-litter categories on drone orthophoto by a multi-class neural network. *Mar. Pollut. Bull.* **169**, 112594 (2021).
12. Xiuling, Z. et al. Starting from the structure: A review of small object detection based on deep learning. *Image and Vision Computing* 105054. (2024). https://doi.org/10.1016/j.imavis.2024.105054 (2024).
13. Santad, T. et al. Application of YOLO deep learning model for real-time abandoned baggage detection. In: *Proceedings of the IEEE Global Conference on Consumer Electronics* (IEEE, 2018).
14. Kim, D., Kim, H., Mok, Y., Paik, J. & HLDNet Abandoned object detection using hand luggage detection network. *IEEE Consum. Electron. Mag.* **11**, 45–56. https://doi.org/10.1109/MCE.2021.3099288 (2021).
15. Su, K. et al. YOLIC: an efficient method for object localization and classification on edge devices. *Image Vis. Comput.* **147**, 105095. https://doi.org/10.1016/j.imavis.2024.105095 (2024).
16. Amudhan, A. N. & Sudheer, A. P. Lightweight and computationally faster hypermetropic convolutional neural network for small size object detection. *Image Vis. Comput.* **119**, 104396. https://doi.org/10.1016/j.imavis.2022.104396 (2022).
17. Liu, H., Liu, Q., Liu, Y. & IEEE International Conference on Image Processing (ICIP). Exploring effective knowledge distillation for tiny object detection. In *Proceedings of the Kuala Lumpur, Malaysia*, (IEEE, 2023), (770–774) https://doi.org/10.1109/ICIP49359.2023.10222589
18. Glenn, J. YOLOv5. GitHub repository (2020). https://github.com/ultralytics/yolov5
19. Ren, S., He, K., Girshick, R., Sun, J. & Faster, R-C-N-N. Towards real-time object detection with region proposal networks. *IEEE Trans. Pattern Anal. Mach. Intell.* **39**, 1137–1149. https://doi.org/10.1109/TPAMI.2016.2577031 (2017).
20. Liu, W. et al. SSD: Single shot multibox detector. In: *European Conference on Computer Vision Amsterdam*, (Springer, 2016). (21–37) https://doi.org/10.1007/978-3-319-46448-0_2
21. Kraft, M. et al. Autonomous, onboard vision-based trash and litter detection in low altitude aerial images collected by an unmanned aerial vehicle. *Remote Sens.* **13**, 929. https://doi.org/10.3390/rs13050929 (2021).
22. Cortesi, I., Mugnai, F., Angelini, R. & Masiero, A. Mini UAV-based litter detection on riverbanks. *ISPRS annals of the photogrammetry, remote sensing and Spatial information sciences* X-4/W1-2022, 87–94 (2023). https://doi.org/10.5194/isprs-annals-X-4-W1-2022-87-2023
23. Yang, J., Tan, K., Liu, S., Zhang, W. & Tao, P. Inshore marine litter detection using radiometric and geometric data of terrestrial laser scanners. *Int. J. Appl. Earth Obs. Geoinf.* **116**, 103180. https://doi.org/10.1016/j.jag.2022.103180 (2023).
24. Chu, Q. et al. Online multi-object tracking using CNN-based single object tracker with spatial-temporal attention mechanism. In: *Proceedings of the IEEE International Conference on Computer Vision Venice*, (IEEE, 2017) (4836–4845) https://doi.org/10.1109/ICCV.2017.516
25. Zhou, X., Koltun, V. & Krähenbühl, P. Tracking objects as points. In: *European Conference on Computer Vision Glasgow* (Springer,2020) (474–490) https://doi.org/10.1007/978-3-030-58592-1_28
26. Shi, P. et al. Monocular 3D object detection algorithm with depth perception capability. *Knowledge-Based Syst. 318*. **3D**, 113539. https://doi.org/10.1016/j.knosys.2025.113539 (2025).
27. Dong, X., Shi, P., Liang, T., Yang, A. & CTAFFNet CNN–Transformer adaptive feature fusion object detection algorithm for complex traffic scenarios. *Transp. Res. Rec.* **2679**, 1947–1965. https://doi.org/10.1177/03611981241231433 (2024).
28. Liu, H., Liu, Q., Liu, Y., Liang, Y. & Zhao, G. Exploring effective knowledge distillation for tiny object detection. In: *Proceedings of the IEEE International Conference on Image Processing (ICIP)* , (IEEE, 2023) (770–774) https://doi.org/10.1109/ICIP49359.2023.10222589
29. Lis, K., Nakka, K., Fua, P., Salzmann, M. & /CVF International Conference on Computer Vision. Detecting the unexpected via image resynthesis. In: *Proceedings of the IEEE* (IEEE, 2019), (2152–2161) https://doi.org/10.1109/ICCV.2019.00224
30. Di Biase, G., Blum, H., Siegwart, R. & Cadena, C. Pixel-wise anomaly detection in complex driving scenes. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (IEEE, 2021) (16918–16927) https://doi.org/10.1109/CVPR46437.2021.01664
31. Nayak, R., Pati, U. C. & Das, S. K. A comprehensive review on deep learning-based methods for video anomaly detection. *Image Vis. Comput.* **106**, 104078. https://doi.org/10.1016/j.imavis.2020.104078 (2021).
32. Vojir, T., Sipka, T., Aljundi, R. & /CVF International Conference on Computer Vision. Road anomaly detection by partial image reconstruction with segmentation coupling. In: *Proceedings of the IEEE* (IEEE, 2021) (15651–15660) https://doi.org/10.1109/ICCV48922.2021.01539

33. Bogdoll, D. et al. Multimodal detection of unknown objects on roads for autonomous driving. In: *Proceedings of the IEEE International Conference on Systems, Man, and Cybernetics (SMC)* (IEEE, 2022), (325–332) https://doi.org/10.1109/SMC53654.2022.9945426
34. St-Charles, P. L., Bilodeau, G. A. & Bergevin, R. A self-adjusting approach to change detection based on background word consensus. In: *Proceedings of the IEEE Winter Conference on Applications of Computer Vision (2015)*, (IEEE, 2015), (990–997) https://doi.org/10.1109/WACV.2015.137
35. Lin, K., Chen, S. C., Chen, C. S., Lin, D. T. & Hung, Y. P. Abandoned object detection via Temporal consistency modeling and back-tracing verification for visual surveillance. *IEEE Trans. Inf. Forensics Secur.* **10**, 1359–1370. https://doi.org/10.1109/TIFS.2015.2408353 (2015).
36. Yun, K. et al. Vision-based litter dumping action detection for real-world surveillance platform. *ETRI J.* **41**, 494–505. https://doi.org/10.4218/etrij.2019-0058 (2019).
37. Sarker, N., Chaki, S. & Das, A. Illegal trash thrower detection based on HOG-SVM for a real-time monitoring system. In: *Proceedings of International Conference on Computing, Communication and Security* (IEEE, 2021).
38. Shyam, D., Kot, A. & Athalye, C. Abandoned object detection using pixel-based finite state machine and single shot multibox detector. In: *Proceedings of the IEEE International Conference on Multimedia and Expo (ICME)*, (IEEE,2018) (1–6) https://doi.org/10.1109/ICME.2018.8486645
39. Din, M., Bashir, A., Basit, A. & Lakho, S. Abandoned object detection using frame differencing and background Subtraction. *Int. J. Adv. Comput. Sci. Appl.* **11**, 541–548. https://doi.org/10.14569/IJACSA.2020.0110770 (2020).
40. Bouwmans, T., Porikli, F., Höferlin, B. & Vacavant, A. *Background Modeling and Foreground Detection for Video Surveillance* (CRC, 2014). https://doi.org/10.1201/b17447
41. Chen, L. C. et al. Encoder-decoder with atrous separable convolution for semantic image segmentation. In: *Proceedings of the European Conference on Computer Vision (ECCV)* (Springer, 2018), (801–818) https://doi.org/10.1007/978-3-030-01234-2_49
42. Pizer, S. M., Johnston, R. E., Ericksen, J. P., Yankaskas, B. C. & Muller, K. E. Contrast-limited adaptive histogram equalization: Speed and effectiveness. In: *Proceedings of the First Conference on Visualization in Biomedical Computing Atlanta*, (IEEE, 1990), (337–345)https://doi.org/10.1109/VBC.1990.109340
43. Jiao, S., Li, X. & Lu, X. An improved Otsu method for image segmentation. In: *Proceedings of the 8th International Conference on Signal Processing* (IEEE, 2006).
44. Jocher, G., Chaurasia, A., Qiu, J. & Ultralytics, Y. O. L. O. (Version 8.0.0) [Computer software]. GitHub (2023). https://github.com/ultralytics/ultralytics
45. Dwibedi, D. et al. Counting out time: Class agnostic video repetition counting in the wild. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (IEEE, 2020), (10387–10396) https://doi.org/10.1109/CVPR42600.2020.01040
46. Bayraktar, E. & ReTrackVLM Transformer-enhanced multi-object tracking with cross-modal embeddings and zero-shot re-identification integration. *Appl. Sci.* **15**, 1907. https://doi.org/10.3390/app15041907 (2025).
47. Yan, B., Peng, H., Fu, J. & /CVF International Conference on Computer Vision. Learning spatio-temporal transformer for visual tracking. In: *Proceedings of the IEEE* (IEEE, 2021). (10448–10457) https://doi.org/10.1109/ICCV48922.2021.01029
48. Rezatofighi, H. et al. Generalized intersection over union: A metric and a loss for bounding box regression. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Long Beach*, (IEEE, 2019). (658–666) https://doi.org/10.1109/CVPR.2019.00075
49. Majchrowska, S. et al. Deep learning-based waste detection in natural and urban environments. *Waste Manage.* **138**, 274–284. https://doi.org/10.1016/j.wasman.2022.02.007 (2022).
50. Amrutha, J. M., Nandini, S. & Anjali, T. Real-time litter detection system for moving vehicles using YOLO. In: *Proceedings of the 4th International Conference on Smart Systems and Inventive Technology (ICSSIT)* (IEEE, 2022). (1311–1315) https://doi.org/10.1109/ICSSIT53264.2022.9716415
51. Nyarko, K., Idowu, I., Ladeji-Osias, K. & Rahman, M. Dual temporal differencing for real-time abandoned object detection. In: *Proceedings of the International Workshop on Artificial Intelligence and Image Processing (IWAIIP) Yogyakarta, Indonesia*, (IEEE, 2023). (69–75) https://doi.org/10.1109/IWAIIP58158.2023.10462839
52. Lis, K. et al. Perspective aware road obstacle detection. *IEEE Rob. Autom. Lett.* **8**, 3245–3252. https://doi.org/10.1109/LRA.2023.3245410 (2023).
53. Krusch, P., Bochinski, E., Eiselein, V. & Sikora, T. A consistent two-level metric for evaluation of automated abandoned object detection methods. In: *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, (4352–4356) (2017).
54. Wojke, N., Bewley, A. & Paulus, D. Simple online and realtime tracking with a deep association metric. In: *Proceedings of the IEEE International Conference on Image Processing (ICIP) IEEE*, (3645–3649). (2017). https://doi.org/10.1109/ICIP.2017.8296962
55. Zhang, Y. et al. ByteTrack: Multi-object tracking by associating every detection box. In: *Proceedings of the European Conference on Computer Vision (ECCV)* ,(Springer, 2022) (1–21). https://doi.org/10.1007/978-3-031-19830-4_1
56. Du, Y. et al. StrongSORT: make DeepSORT great again. *IEEE Trans. Multimedia*. https://doi.org/10.1109/TMM.2023.3264215 (2023).
57. Maggiolino, G., Ahmad, A., Cao, J. & Kitani, K. Deep OC-Sort: Multi-pedestrian tracking by adaptive re-identification. In *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, (IEEE, 2023). (3025–3029) https://doi.org/10.1109/ICIP49359.2023.10222576
58. Stanojević, V., Todorović, B. & BOOSTTRACK++ Using tracklet information to detect more objects in multiple object tracking. Preprint at https://arXiv/org/240813003 (2024).
59. Lee, J. G., Han, J. & Whang, K. Y. Trajectory clustering: A partition-and-group framework. In: *Proceedings of the ACM SIGMOD International Conference on Management of Data ACM*. (2019). https://doi.org/10.1145/3299869.3319860

Author contributions

Dian Liu proposed the main idea and implementation.Dian Liu and Yang Liu wrote the main manuscript text. Haoxiang Wang and Xingming Zhang contributed to method development.Xiujie Zhang contributed to the dataset establishement.All authors reviewed the manuscript.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to Y.L.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2025